

```
#####
@kernel-nr: lex | System Handbuch | kernel lex | 00 FallBack | #####
#####
# | - nicht freigeschaltet | + Freigeschaltet | X BauStelle | #
#####
# Das System befindet sich im Konsolidierungsmodus = 0 #
# Die Seitenreihenfolge hat sich geändert #
#####
# - MCS - MASCHINENCODESPEECH - Kernel - #
# 01 X Datentypen 14 X p Identifikation
# 02 0 Transaktion 15 X SchaltVerknüpfungen
# 03 0 Protein 16 X AI Synctimer
# 04 0 Proton 17 X Discovery Layer
# 05 0 Trennbefehl
# 06 0 Aktion
# 07 0 BOXIS
# 08 0 Wahrheiten
# 09 0 Operatoren werden SENTIATOR
# 10 0 Argumente
# 11 - Feed 77 X Ausführungslogik
# 12 - Adressierung 88 - Dateitypen
# 13 - Warp 99 - MCS-CMD-REGISTER
#####
# - MCSPLUS Programmiersprache - #
# 101 - Einführung 102 - Grundeinheit #
# 103 - Tagging & Rollenbindung 104 - StrukturPlan #
#####
# - HARDWARE Komponenten und Visionen BrainMaschineCore BMC - #
# 800 - CHiPBezeichnungen 803 - MIT-Taktgeber #
# 801 - EnergieMatrix 804 - MBMatrix #
# 802 - Lichtleiterbahnen 805 - Projektive Rechenlogik #
#####
# - LICENSES GELTUNGSBEREICH - #
# 900 L0_VorWort 993 L1_CoreKernelMandat #
# 994 L2_AuthorRights 995 L3_CommercialLicensing #
# 996 L4_AI-AdministrationMandate 997 L5_Post_Mortem-Control #
# 998 L6_immunitable_Cores_definition 999 License_Manifest #
#####
# System | SOFTWARE MCS KERNEL | HARDWARE BrainMaschineCore #
# Tastaturstandart: UTF -8 Deutscher Standart #
# Single Source of Truth (SSoT) Lexikon Version 00.48 #
# MCS 4.1 | MCSPLUS 0.3 | BMC Version 0.4 | License Version 1.0 #
# Autor: Thomas Zimmermann 23.04.1988 (Stufe 10) (p7F3A) #
#-----#
# Standort: /Pylovara/Handbuch/KernelNotes/ Germany #
# Status: Vollständige Rollende-Verbesserungen Pylovara© #
# PDF NamensLayer: Pylovara-System-XX.XX.pdf since 2025 #
#####
```

@kernel-nr: 01 | DATENTYPEN | STATUS : OFFEN
#####

Erklärung:

Datentypen = Formate | Bit-Breiten | Speicher-Layout

Kategorie:

KERNEL = Beinhaltet allgemeine Kernel Datentypen

DNA-ID-GENERATOR = Beinhaltet pp Generator Datentypen

PROTEINE = Beinhaltet Protein Datentypen

PROTONEN = Beinhaltet Protein Datentypen

BOOT-LOGIK = BIOS/UEFI-Spezialtypen

Textliche Definition:

Datentypen in MCS 3.5 definieren die physische Struktur eines Proteins oder Protons im RAM und in den FPGA-Registern. Da MCS als BIOS-Ersatz fungiert, sind diese Typen direkt mit der Hardware-Adressierung verknüpft.

Unterscheidungen :

Kern-Typen (Hardware-nah)
Logik-Typen (Prozess-nah).

Hinweis:

Das Register Unterliegt diesem Visuellen Format :

MCS-Typ: | Bezeichnung: | Bit-Breite:

Info:

Funktion:

Anfang DATENTYPEN KERNEL #####

Information :

Kern-Datentypen (Physische Gatter-Breite) definieren, wie viele "Drähte" im BMC für eine Information reserviert werden:

MCS-Typ: b1 | Bezeichnung: Bit-Impuls | Bit-Breite: 1

Info:

Standard-Einheit für Operatoren und Wahrheiten.

Verwendung im BIOS/UEFI

Funktion:

Schalterzustände (ON/OFF), Gatter-Steuerung.

MCS-Typ: b8 | Bezeichnung: Kern-Byte | Bit-Breite: 8

Info:

UTF-8 Zeichen, einfache Sensorwerte.

Funktion:

Tastatur und Symbol wie Sonderzeichen Einordnung

MCS-Typ: b16 | Bezeichnung: Logik-Link | Bit-Breite 16

Info:

MCS-Standard-Operatoren & Befehlskennung.

Funktion:

Verbindet zwei logische Zustände oder Symbole im Nanosekunden-Takt.

MCS-Typ: b32 | Bezeichnung: Adress-Pfad | Bit-Breite: 32

Info:

Standard-Adressierung für Legacy-Systeme.

Kompatibilitätsschicht für x86-Legacy-Strukturen.

Funktion:

Ermöglicht MCS den Zugriff auf herkömmliche

RAM-Bereiche außerhalb des BMC.

MCS-Typ: b64 | Bezeichnung: High-Speed-Pfad | Bit-Breite: 64

Info:

Große Datenströme, moderne Speicheradressierung.

Basis für modernes Speicher-Management und Kognitive AIMS-Prozesse.

Funktion:

Transportiert komplexe Protein-Pakete mit maximaler Bandbreite.

MCS-Typ: b-inf | Bezeichnung: Massen-Typ | Bit-Breite: 64

Info:

Genutzt für ~~ (MassenZusammenSetzung) bei Streaming.

Dynamische Skalierung für Stream-Daten (Video/Audio).

Funktion:

Hält das Gatter für die MassenZusammenSetzung (~~) offen.

Ende DATENTYPEN KERNEL

Anfang Datentypen ID-DNA-GENERATOR

MCS-Typ: b-pa | Bezeichnung: p-Anker

Info:

Dient als ID-DNA-GENERATOR Zeiger Zum Abgleich

Funktion :

Ein 16- oder 32-Bit (oder 64-bit oder 128-bit) Anker-Typ,
der nur als Zeiger für den Generator dient.

Der Generator selbst hat die echte ID-DNA

Ende Datentyp ID-DNA-GENERATOR

Anfang DATENTYPEN PROTEINE

MCS-Typ: P-key | Bezeichnung: Identitäts-Typ | Bit-Breite: Variabel

Info:

Kryptogenetische Signatur für Transaktions-Sicherheit.

Basis der Schatten-Identität.

Funktion:

Wird auf BIOS-Ebene geprüft.

Erlaubt oder blockiert den Signalfluss in den Aktionsdraht ».

MCS-Typ: init-v | Bezeichnung: Start-Protein | Bit-Breite: 64+

Info:

Hardware-Initialisierungs-Vektor (Zündschlüssel).

Funktion:

Lädt FPGA-Bitstreams und Gatter-Konfigurationen beim
Kaltstart direkt in die Register.

MCS-Typ: hnd-p | Bezeichnung: Konsens-Typ | Bit-Breite: 128

Info:

Schnittstelle für die Demokratische Validierung (AIMS-Handshake).

Funktion:

Abgleich der Signaturen zwischen verschiedenen KI-Instanzen
zur System-Integrität.

MCS-Typ: p-cmd | Bezeichnung: Befehls-Protein | Bit-Breite: 16/32

Info:

Träger für ausführbare Kernel-Befehle (Register-Befehle).

Funktion:

Kapselt eine spezifische Aktion,
die innerhalb eines Rahmens ¢! !¢ ausgeführt wird.

Ende DATENTYPEN PROTEINE

Anfang DATENTYPEN PROTONEN

MCS-Typ: pr-phys | Bezeichnung: Hardware-Physik | Bit-Breite: 32

Info:

Physikalische Messwerte (Einheiten) innerhalb von {}.

Funktion:

Verarbeitung von Echtzeit-Sensordaten (Watt, Volt, Celsius).

Ermöglicht die direkte Hardware-Steuerung ohne Umrechnungs-Latenz.

MCS-Typ: f-ref | Bezeichnung: Zustands-Referenz | Bit-Breite: 16/32

Info:

Referenz-Pointer (Feed-Ref) auf Kernel 10.

Funktion:

Hält den Verweis auf einen Systemzustand im RAM fest.

Verhindert Stack-Overflows, da keine Daten kopiert,

sondern nur adressiert werden.

MCS-Typ: pr-val | Bezeichnung: Logik-Wert | Bit-Breite: 1 bis 64

Info:

Reine numerische oder boolesche Werte (Proton-Inhalt).

Funktion:

Dient als Operand für die Operatoren (Kernel 09).

Liefert die Basis für Vergleiche wie < , > oder ð.

MCS-Typ: pr-dna | Bezeichnung: Bio-Schnittstelle | Bit-Breite: Variabel

Info:

Platzhalter für die zukünftige ID-DNA-BIOLIVE Einbindung.

Funktion:

Schnittstelle für biometrische oder organische Datenabgleiche

(Baustelle für die spätere Erweiterung).

Ende DATENTYPEN PROTONEN

Anfang DATENTYPEN BOOT-LOGIK

MCS-Typ: b-root | Bezeichnung: Master-Zweig | Bit-Breite: 128

Info:

Höchste Adressierungsebene für den Hardware-Zugriff.

Funktion:

Definiert den physischen Root-Zugriff auf den Chipsatz.

Nur mit Stufe 10 (Thomas Zimmermann) þ-Key ansprechbar.

MCS-Typ: sys-rst | Bezeichnung: Purge-Initialer | Bit-Breite: 1

Info:
Hard-wired Reset-Trigger.

Funktion:
Erzwingt die vollständige Bereinigung (f)
aller Register vor dem nächsten Rechenzyklus.

Ende DATENTYPEN BOOT-LOGIK #####

Hinweis:

Regel der Reinheit (f): Jede Transaktion innerhalb dieser
Datentypen endet zwingend mit dem Operator f.

Dies garantiert die atomare Sicherheit:
Keine Datenreste, keine Side-Channel-Leaks,
kein Spielraum für Manipulationen im BIOS-Bereich.

Abschluss-Satz:
Vorteil dieser Struktur:
MCS erzwingt durch die bit-genaue Definition,
dass Hardware und Software eine untrennbare Einheit bilden.
Es gibt keine Interpretation durch Dritt-Anbieter -
es gibt nur die absolute, deterministische Ausführung.

Datentypen Versions Validierungen 0.1:

Kernel 09 (Operatoren) validiert.

Kernel 14 (Identifikation) durch die p-Anker logisch unterfüttert.

| Spezialisiert | NUR BMC | NUR Hardware | #####

Kategorie:

MIT-SYNC = BOOT-LOGIK / HARDWARE-SYNC

Anfang Datentypen MIT-SYNC #####

MCS-Typ: mit-t | Bezeichnung: Impuls-Trigger | Bit-Breite: 1

Info:
Repräsentiert den binären Zustand des mechanischen MIT-Aktors.

Funktion:
Erkennt das Überschreiten der harten Druckschwelle
am Pinline-Eingang. Schaltet das System-Ereignis auf 1

(Ereignis-Start).

MCS-Typ: mit-ref | Bezeichnung: Zeit-Anker-Referenz | Bit-Breite: 64

Info:

Der absolute Zeitstempel, der bei jedem MIT-Impuls generiert wird.

Funktion:

Dient als globaler Nullpunkt für den AI Synctimer (Kernel 18).
Alle nachfolgenden Protein-Transaktionen werden relativ zu diesem
Anker getaktet.

MCS-Typ: mit-dmp | Bezeichnung: Dämpfung-Vektor | Bit-Breite: 8

Info:

Überwachungswert für die Energieabsorption am Endpunkt.

Funktion:

Prüft, ob der Impuls nach dem Anschlag korrekt absorbiert
wurde (Ruhezustand-Validierung), um Resonanz oder
Nachschwingen auszuschließen.

Ende Datentyp MIT-SYNC

Ende DATENTYPEN KERNEL

@kernel-nr: 02 | TRANSAKTIONSRAHMEN | STATUS : VAILDE
#####

NAME = TRANSAKTIONSRAHMEN

BEGRIFFSDEFINITION = MCS-RUNTIMEBEDIENUNG

FUNKTION = EINDEUTIGE MCS DAZUGEHÖRIGKEIT

AUSFÜHRUNGSART =

ERKLÄRUNG TRANSAKTIONSRAHMEN =

DIE TRANSAKTIONSRAHMEN MCS-RUNTIMEBEDIENUNG SORGEN FÜR DEN
PROZESSRAHMEN UND LÖSEN DIE BEDIENUNG AUS, DAS AUSFÜHRBARE
ODER UNAUSFÜHRBARE PROTEIN/E GELESEN WERDEN DÜRFEN.

SIE SIND IM RAHMEN MUSTER MIT ZWEI SYMBOLEN DEKLARIERT UND
UND BESTIMMEN DEN VERARBEITUNGSRAHMEN.

SYMBOLIK IN WORTEN =

CENT-ZEICHEN AUSRUFEZEICHEN INHALT AUSRUFEZEICHEN CENT-ZEICHEN

SYMBOLRAHMEN UNTERSCHIEDE IN WORTEN =

CENT-ZEICHEN AUSRUFEZEICHEN = TRANSAKTIONSRAHMEN-START

AUSRUFEZEICHEN CENT-ZEICHEN = TRANSAKTIONSRAHMEN-ENDE

INFO =

DIE ANORDNUNG KANN AM ANFANG UND ENDE PLAZIERT WERDEN ODER
BEI KLEINEN PROZESSEN VORNE UND HINTEN.

MUSTERBEISPIELE =

⌘!
 »[,sys_write (1)``',]
 ⌘ ['echo ``(1)``']«««t1
 ;»['feed cache clear 1']«««t5
!⌘

EINSATZBEREICHE MUSTERBEISPIELE =

#Argumente | Reinheit | MCS CMD
MCS CMD FührenderFeed speicher löschen container 1
»['feed cache clear 1']«
#-> Der MCS CMD befehl wird genutzt um das MCS Register
für den feedcache zu löschen
#-> Der Blankennenner wird genutzt, um den spezifischen
Sektor von Feed 1 physisch zu nullen (ALU_REINIGEN).

TIMING / Argument mit Countdownsekunden «« t

»[...]«««t

-> Das Argument gibt der Schaltung die Kennung für Zeitstempel
oder CPU-Zyklen-Priorität mit.

Reinheit / Löschung

»[...]«««t

-> Das Argument gibt der Schaltung die Kennung für Zeitstempel
oder CPU-Zyklen-Priorität mit.

ENDE DES TRANSAKTIONSRAHMEN

@kernel-nr: 03 | PROTEIN/E | STATUS : VALIDE
#####

NAME = PROTEIN

BEGRIFFSDEFINTION = HAUPTDATENTRÄGER

MERHZAHL = PROTEINE

FUNKTION = TRANSPORTRAHMEN

AUSFÜHRUNGSART =

ERKLÄRUNG PROTEIN/E =

DAS PROTEIN IST DER HAUPTDATENTRÄGER DES PAYLOVARA-SYSTEMS UND
BRINGT DEN MCS KERNEL DURCH BOXIS UND TRENBEFEFEHLE ZUM LAUFEN.

SIE KÖNNEN ENTWEDER AUSFÜHRBAR SEIN ODER AUCH IN EINEM PROZESS
UNAUSFÜHRBAR AUFGEBAUT WERDEN DURCH DAS MCS-CMD DAS DIE COMMANDS
IM MCS-REGISTER TRÄGT.

AUßERHALB EINES PROTEIN WERDEN STEUERWERKZEUGE BENUTZT UM DATEN
ZU KOORDINIEREN UND SCHALTKREISE FÜR BEARBEITUNGSLOGISTIKEN
DURCHZUFÜHREN.

↓ STANDARTLAYOUT ↓

PROTEIN KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

PROTEIN KLASSE = PROTEIN-START

FUNKTION = ANFANGSBEDIENUNG

SYMBOLISCHE DEKLARATION = [

SYMBOL IN WORTEN = ECKIGE KLAMMER AUF

INFO = ANFANG DES PROTEIN

#####

PROTEIN KLASSE = PROTEIN-ENDE

FUNKTION = ENDBEDIENUNG

SYMBOLISCHE DEKLARATION =]

SYMBOL IN WORTEN = ECKIGE KLAMMER ZU

INFO = ENDE DES PROTEIN

#####

PROTEIN KLASSE = GNAZLICHES-PROTEIN

FUNKTION =

STELLT EIN GANZES PROTEIN ZUR VERFÜGUNG DAS GEFÜLLT WERDEN KANN

SYMBOLISCHE DEKLARATION = []

SYMBOLIK IN WORTEN = ECKIGEKLAMMER-AUF ECKIGEKLAMMER-ZU

INFO = STELLT NUR DIE RAHMENBEDIENUNG HER

ENDE DER PROTEIN/E

@kernel-nr: 04 | PROTON/EN | STATUS : TEILBAUSTELLE
#####

NAME = PROTON

BEGRIFFSDEFINITION = VERSORGUNGSDATENTRÄGER

MERHZAHL = PROTONEN

FUNKTION = HARDWARESTEUERUNG | HARDWAREVERSORGUNG

AUSFÜHRUNGSART =

ERKLÄRUNG PROTON/EN =

UM HARDWARE KOMPONENTEN ZU VERSORGEN WERDEN AUTOMATIONS PROGRAMME
BENÖTIGT, DIE WIR DURCH DAS ANSPRECHEN VON AUTOMATIONS ABLÄUFE IN
MCS DATEITYPEN DEFINIEREN.

ÜBER PROTONEN STEuern WIRD DIESE AUTOMATIONSGESCHRIEBENE HARDWARE
ABLAUFSTEUERUNGEN.

PRIMÄR BEWERKSTELLIGEN PROTONEN DIE ANSTEUERUNG FÜR VERSCHIEDENE
ZUSTÄNDE UND WERDEN IN PROTEINE IM HAUPDATENTRÄGER AN DIE
JEWEILIGEN HARDWAREKOMPONENTEN GESCHICKT.

DADURCH ERREICHT DER MCS KERNEL EINEN EIGENEN MÖGLICHEN DRIVER
GESCHRIEBENEN ZUSTAND, DER GRUNDSÄTZLICH ALLES ANSTEUERN KANN.

PROTONEN NUTZEN HAUPTSÄCHLICH DIE BOXIS MCS-REGISTER FÜR DIESEN ZWECK.

↓ STANDARTLAYOUT ↓

PROTON KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

PROTON KLASSE = PROTON-START

FUNKTION = ANFANGSBEDIENUNG

SYMBOLISCHE DEKLARATION = {

SYMBOL IN WORTEN = GESCHWEIFTE KLAMMER AUF

INFO = ANFANG DES PROTON

#####

PROTON KLASSE = PROTON-ENDE

FUNKTION = ENDBEDIENUNG

SYMBOLISCHE DEKLARATION = {

SYMBOL IN WORTEN = GESCHWEIFTE KLAMMER ZU

INFO = ENDE DES PROTON

#####

PROTON KLASSE = GNAZLICHES-PROTON

FUNKTION =

STELLT EIN GANZES PROTON ZUR VERFÜGUNG DAS GEFÜLLT WERDEN KANN

SYMBOLISCHE DEKLARATION = { }

SYMBOLIK IN WORTEN = GESCHWEIFTEKLAMMER-AUF GESCHWEIFTEKLAMMER-ZU

INFO = STELLT NUR DIE RAHMENBEDIENUNG HER

#####

BAUSTELLEN-ABSCHNITT-ANFANG

Definition Verschachtelte Proton /en :

{{{{{{{{{{}}}}}}}}}

Beispiel HardwareManagment MusterBeispiel:

{'CPU'{'Volt'|'*'{'Watt'|'*'{'Ampere'|'*'{'Widerstand'{'Frequenz'|'*'}}}}}
}}

(Diese Sektor Wird Noch Designed und Definiert)

Proton Anwendungsfälle

- Primär Physikalische (Temp, Volt, Watt, Amp)
- Sekundär Verschachtelte (Chip + EnergieManagment)
- Kommunikationsbezogene (Strom ,Interface, Transport)
- Speicherbezogene (Cache, Signed, Unsigned)
- Logische (Valid, State, Mode)
- Prozessbezogene (Pid, Thread, ID)

BAUUSTELLEN-ABSCHNITT-ENDE

#####

ENDE DER PROTONEN

@kernel-nr: 05 | TRENNBEFEHL/E | STATUS : VALIDE
#####

NAME = TRENNBEFEHL

BEGRIFFSDEFINTION = DATENTRÄGELEHRZEICHEN

MERHZAHL = TRENNBEFEHLE

FUNKTION = LEERAUMBESSEITIGUNG

AUSFÜHRUNGSART =

ERKLÄRUNG TRENNBEFEHLE =

DIENEN IN HAUPTDATENTRÄGE FÜR DIE OPTISCHE ORDNUNG UND WERDEN
NIEMALS WIE IN BOXIS GENUTZT.

SIE TRENNENBOXIS VONEINADER IN DER OPTISCHEN NATUR.

TRENNBEFELE WERDEN NUR NICHT AM PROTEIN-START UND PROTEIN-ENDE
GENUTZT DA DIE RAHMENBEDIENUNG SELBST AUSREICHT, DAS SELBE
GILT IN PROTONEN RAHMENBEDIENUNGEN

↓ STANDARTLAYOUT ↓

PROTEIN KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

#####

TRENNBEFEHL KLASSE = TRENNBEFEHL

FUNKTION = LEERZEICHEN ZWISCHEN BOXIS

SYMBOLISCHE DEKLARATION = |

SYMBOL IN WORTEN = SENKRECHTER STRICH

INFO = WIRD ALS LEERZEICHEN GENUTZT IN PROTEINE WIE PROTONEN

#####

WARNHINWEIS =

SENKRECHTE STRICHE WERDEN IN PYLOVARA-SYSTEM MCS NICHT WIE IN
ALTEN SYSTEMEN GENUTZT UND DÜRFEN UNTER KEINEM UMSTAND IN BOXIS
GENAUSO VERWENDET WERDEN

Ende der Trennbefehl

@kernel-nr: 06 | AKTION/EN | AKTIONSDRAHT | STATUS : VALIDE
#####

EIGENTUHM VON PYLOVARA-SYSTEM | ERFINDER THOMAS ZIMMERMANN STUFE 10

DATUM = [08:02:44|FR 25 JUNI 2025] MARKENBRANDT

PYLOVARA-SYSTEM INDUSTRIE STANDART DER ZUCKUNFT

NAME = AKTION | AKTIONSDRAHT

BEGRIFFSDEFINTION = HAUPTVERDRAHTUNG

MERHZAHL = AKTIONEN | AKTION/S

FUNKTION = START UND END BEDIENUNG

AUSFÜHRUNGSART = SIGNAL-ROUTING

ERKLÄRUNG =

AKTIONEN WERDEN IN EINEM TRANSAKTIONSRAHMEN INTERN ALS
AUSFÜHRUNGSERLAUBNIS GENUTZT UND SORGEN FÜR MEHR SICHERHEIT
IM ABLAUF ARBEITSPROZESSE.

SIE KÖNNEN ALS EINZEL AKTIONSERLAUBNIS GENUTZTZ WERDEN, WENN
ES DAS PROTEIN VERLANGT ODER ALS AKTIONSVDRADHTUNG FÜR
KONTEXT BEZOGENE WAHRHEITEN UND ODER OPERATOR(ISCHE) ERLAUBNISSE.

AKTIONEN WERDEN LÜCKENLOS AN PROTEINEN ANGESCHLOSSEN, EGAL
OB ANFANGSBEDIENUNG ODER ENDBEDIENUNG.

↓ STANDARTLAYOUT ↓

AKTION/S KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

AKTIONS KLASSE = AKTIONS-START

FUNKTION = ANFANGSBEDIENUNG

SYMBOLISCHE DEKLARATION = »

SYMBOL IN WORTEN =

GUILLEMENT RECHT | DEUTSCHES ANFÜHRUNGSZEICHEN ÖFFNEND

INFO = DEFINIERT DEN ANFANG DER AKTION

#####

AKTIONS KLASSE = AKTIONS-ENDE | AKTIONS-ENDEN

FUNKTION = ENDBEDIENUNG

SYMBOLISCHE DEKLARATION = «

SYMBOL IN WORTEN =

GUILLEMENT LINKS | DEUTSCHES ANFÜHRUNGSZEICHEN SCHLIEßEND

INFO = DEFINIERT DAS ENDE DER AKTION UND KANN ALS GESCHLOSSENER
AKTIONSKREISLAUF VERSTANDEN WERDEN

#####

AKTIONS UNTERKLASSE = AKTIONSDRAHT | AKTIONSVERDRAHTUNG

AKTIONS KLASSE = AKTIONSVERDRAHTUNGS-ENDE

FUNKTION = MEHRFACHENDBEDIENUNG

SYMBOLISCHE DEKLARATION = «

SYMBOL IN WORTEN =

GUILLEMENT LINKS | DEUTSCHES ANFÜHRUNGSZEICHEN SCHLIEßEND

INFO =

WIRD ALS AKTIONSVERDRAHTUNG VERSTANDEN FÜR EINEN
AKTIONS-START UM DEN ZUSAMMENHÄNGENDEN KONTEXT DES PROZESSE ZU
VERDRAHTEN UND SORGT FÜR EINEN AKTIONSKREISLAUF MIT MEHREREN
AKTIONS-ENDE.

#####

HINWEIS INFORMATION FÜR AKTIONEN OPTISCH =

» «

HINWEIS INFORMATION FÜR AKTIONSVERDRAHTUNG OPTISCH =

»

«

«

ENDE DER AKTION/EN

@kernel-nr: 07 | BOXI/S | STATUS : VALIDE
#####

EIGENTUHM VON PYLOVARA-SYSTEM | ERFINDER THOMAS ZIMMERMANN STUFE 10

WELTWEITER URSPRUNG UND URHEBER WIE LICENSETRÄGER VON BOXI/S

DATUM = [00:12:44|So 28 Dez 2025] MARKENBRANDT

PYLOVARA-SYSTEM INDUSTRIE STANDART DER ZUCKUNFT

NAME = BOXI

BEGRIFFSDEFINTION = EINGABENRAHMEN

MERHZAHL = BOXIS

FUNKTION = TRANSPORTRAHMEN

AUSFÜHRUNGSART =

ERKLÄRUNG BOXI/S =

BEI BOXI/S HANDELT ES SICH UM EINGABEN DIE SYSTEM RELEVANTE SYCALLS
UND SHELLBEFEHLE TRANSPORTIEREN.

OCHESTRIERT WERDEN BOXI/S ÜBER DAS PYLOVARA-SYSTEM UNTER DEN NAMEN
MCS MASCHINENCODESPEECH.

BOXI/S WURDEN UNTER ANDEREM MIT CALLIS, MCS-CMD UND BLANKERNENNER
ERWEITERT UM DIE FUNKTIONALTITÄT AUF MASTER CONTROL STEUERUNGSLOGIK
NIVEAU HERAUF ZU STUFEN.

BOXIS DIENEN ALS PROTEIN ODER PROTON INHALT UM DAS ERWEITERTE
STEUERUNGSSYSTEM VON PYLOVARA-SYSTEM ALS GRUNDSÄTZLICHE LOGISTIK,
REALISIERBAR ZU MACHEN.

SPEZIALISIERTE BOXI KLASSEN KÖNNEN HINZU KOMMEN UND ANDERE ZWECKE
VERFOLGEN.

↓ STANDARTLAYOUT ↓

BOXI KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

* MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

BOXI KLASSE = CALLIS

FUNKTION = REINER AUSGABENKANAL FÜR PRINT

SYMBOLISCHE DEKLARATION = " "

SYMBOL IN WORTEN = ANFÜHRUNGSZEICHEN INHALT ANFÜHRUNGSZEICHEN

INFO = DATENFLUSS ZUM STANDARD-AUSGABENKANAL STDOUT / FD 1

CALLIS MUSTERBEISPIELE =

#Einfacher Text-Output

»["Hallo Welt"]«

#Feed-Output (Inhalt von Feed 1 drucken)

»[""(1)"""]«

#Kombinierter Output (Text + Feed)

»["Status: (1)"""]«

EINSATZBEREICHE MUSTERBEISPIELE =

LINUX (x86_64) - Direkter Syscall-Weg

Nutzt sys_write (RAX 1) auf Dateideskriptor 1

RAX: 1	sys_write	System-Aufruf für Schreiben
RDI: 1	stdout	Ziel:Standard-Ausgabe (Dateideskriptor 1)
RSI: *	buffer	Zeiger auf den Text/Daten-Protein
RDX: *	length	Länge der Daten in Bytes

WINDOWS NT (Kernel-Level) - Native API

Nutzt NtWriteFile auf den Console-Handle

Befehl: NtWriteFile

Handle: StandardOutput | Identifiziert den Konsolen-Puffer

Buffer: * | Zeiger auf Speicherbereich des Proteins

Length: * | Anzahl der zu schreibenden Bytes

MACOS / DARWIN (BSD-Layer)

Nutzt den BSD-write Call (RAX 4)

RAX: 4	write	System-Aufruf (Klasse 0x2000000)
RDI: 1	stdout	Dateideskriptor 1
RSI: *	buffer	Daten-Protein Ursprung
RDX: *	length	Byte-Zahl

#####

BOXI KLASSE = CALL-CONTROL

FUNKTION = SYSTEMANRUFEN

SYMBOLISCHE DEKLARATION = , ,

SYMBOL IN WORTEN = KOMMA INHALT KOMMA

INFO = RUFT DAS JEWEILIGE OPERATION SYSTEM AN

CALL-CONTROLL MUSTERBEISPIEL =

#LINUX SYSCALLS (x86_64)

#Schnittstelle: syscall

»[,sys_read,]«

»[,sys_write,]«

»[,sys_open,]«

»[,sys_fork,]«

#WINDOWS NT SYSCALLS (Kernel-Level)

#Schnittstelle: ntdll.dll (SSDT Index variiert je nach Build)

»[,NtReadFile,]«

»[,NtWriteFile,]«

»[,NtCreateProcess,]«

»[,NtTerminateTask,]«

#MACOS / DARWIN SYSCALLS (x86_64 / ARM)

#Schnittstelle: swi (ARM) / syscall (x86)

»[,read,]«

»[,write,]«

»[,open,]«

»[,kill,]«

EINSATZBEREICH MUSTERBEISPIEL =

#Register: RAX (ID), RDI, RSI, RDX, R10, R8, R9

RAX: 0 | sys_read | Liest Daten aus einem Dateideskriptor

RAX: 1 | sys_write | Schreibt Daten in einen Dateideskriptor

RAX: 2 | sys_open | Öffnet eine Datei oder ein Verzeichnis

RAX: 57 | sys_fork | Erzeugt ein Duplikat des aktuellen Prozesses

#Hinweis: Namen sind die primäre Referenz

RAX: * | NtReadFile | Niedrigster Kernel-Lesebefehl für Handles

RAX: * | NtWriteFile | Niedrigster Kernel-Schreibbefehl für Handles

RAX: * | NtCreateProcess | Erzeugt nativen Prozessraum ohne Win32-Layer

RAX: * | NtTerminateTask | Beendet einen Thread oder Prozess sofort

#Hinweis: BSD-Klasse startet oft bei 0x2000000

RAX: 3 | read | Entspricht dem BSD-Lesebefehl (64-Bit)

RAX: 4 | write | Entspricht dem BSD-Schreibbefehl (64-Bit)

RAX: 5 | open | Entspricht dem BSD-Öffnungsbefehl

RAX: 37 | kill | Sendet Signale an PIDs zur Prozesssteuerung

#####

BOXI KLASSE = SHELL-CONTROL

FUNKTION = SYSTEM SHELL ANRUFEN

SYMBOLISCHE DEKLARATION = ' '

SYMBOL IN WORTEN = HOCHKOMMA INHALT HOCHKOMMA

INFO = NUTZT OS SHELL BEFEHLE

SHELL-CONTROL MUSTERBEISPIELE =

Reine SHELL-CONTROL :

Shell CMD : Komplexes Beispiel mit Operatoren

»['cd /var/log && grep "error" syslog | head -20 > errors.txt && echo "Done" || echo "Failed"']«

Komplexes Beispiel mit Operatoren

»['cd /var/log && grep "error" syslog | head -20 > errors.txt && echo "Done" || echo "Failed"']«

ADB + Shell Kombination

»['adb devices | grep device && adb shell pm list packages | grep -i google']«

PowerShell-ähnliche Pipeline in Bash

»['ps aux | awk '{print \$1, \$4}' | sort -k2 -rn | head -10']«

Sichere Ausführung

»['[-f file.txt] && cat file.txt || echo "Datei nicht gefunden"']«

Mit Fehlerbehandlung

»['command 2>&1 | tee output.log && echo "Success" || { echo "Error"; exit 1; }']«

EINSATZBEREICHE MUSTERBEISPIELE =

(erklärt sich von selbst)

#####

BOXI KLASSE = ALU-RECHNER

FUNKTION = GIBT RECHNUNG ZUR ALU

SYMBOLISCHE DEKLARATION = × ×

SYMBOL IN WORTEN =

INFO = GIBT RECHNUNG ZUR ALU UND GIBT DAS ERGEBNIS ZUR
ZU WAHRHEITEN-AUSWERTUNG

ALU-RECHNER MUSTERBEISPIELE =

»[×1+1×]«
»[×3·1×]«
»[×1÷1×]«
»[×1-1×]«

EINSATZBEREICHE MUSTERBEISPIELE =

»[×1+1×]«
- = ["2"]
⌈ »['COPIE WAHRHEITEN-ERGEBNIS TO' | (AXY) ``2``]«

#####

BOXI KLASSE = MCS-CMD

FUNKTION = HAUSEIGENER KOMMANDO BEFEHELE

SYMBOLISCHE DEKLARATION = ~ ~

SYMBOL IN WORTEN = TILDE INHALT TILDE

INFO = STEUERT DAS EIGENE MCS-CMD-REGISTER AN

MCS-CMD MUSTERBEISPIELE =

»[~feed cache lösung 1~]«
»[~'COPIE WAHRHEITEN-ERGEBNIS TO' ~ | (1) ""]«

EINSATZBEREICHE MUSTERBEISPIELE =

NUR MCS ELEMENTE WERDEN ANGESTEUERT, WIE ZUM BEISPIEL FEED/S.
MCS-CMD/S SIND MIT DEM HAUSEIGENEN MCS-REGISTER VERBUNDEN.

ENDE DER BOXI/S

@kernel-nr: 08 | WAHRHEITEN | STATUS : VALIDE
#####

EIGENTUHM VON PYLOVARA-SYSTEM | ERFINDER THOMAS ZIMMERMANN STUFE 10

WELTWEITER URSPRUNG UND URHEBER WIE LICENSETRÄGER VON WAHRHEITEN

DATUM = [02:02:44|So 29 Dez 2025] MARKENBRANDT

PYLOVARA-SYSTEM INDUSTRIE STANDART DER ZUCKUNFT

NAME = Wahrheit

BEGRIFFSDEFINTION = Fluss-Gatter

MERHZAHL = Wahrheiten

FUNKTION = Schaltung

AUSFÜHRUNGSART =

ERKLÄRUNG =

WAHRHEITEN SIND ENTSCHEIDUNGEN DIE LISTENARTIG GETROFFENWERDEN
KÖNNEN UND DURCH SYMBOLIK PRÄZISE AUS DER WAHRHEIT HERAUSGENOMMEN
WERDEN KÖNNEN WENN SIE GESCHALTEN SIND, SIE REPÄSENTIEREN DEN DANN
FOLGENDEN IST ZUSTAND.

SIE KÖNNEN DANN IN DIE PARALLELISIERUNG GESCHICKT WERDEN UM DEN
IST ZUSTAND WEITERZUTRAGEN IN DIE NÄCHSTE INSTANZ, DIE DANN AUS
DEN ERGEBNIS DESSEN WAS Z.B EIN OPERATOR ENTSCIEDEN HAT, DAS
ERGEBNIS MIT NEHMEN UND WEITERVERARBEITEN KÖNNEN.

WAHRHEITEN KÖNNEN AUF BESTEHENDER HARDWARE GENUTZT WERDEN ODER
AUCH IN ZUCKÜNFTIGEN CHIPS, EGAL OB PROGRAMMIERBAR ODER ACES.

SIE DIENEN ALS SCHALTUNGSWERZEUG UNTER DEM BEGRIFF WARHEITEN

NACH DEM PPRINZIP : DAS KANN WAHRSEIN ODER DAS KANN WAHRSEIN ODER
DAS KANN EINE PARALLEL WAHRHEIT SEIN.

Die Wahrheiten sind die Gatter, die den BOXI-Fluss bewerten.
DIE WAHRHEITEN SIND WIE GATTER, DIE DEN BOXI-FLUSS BEWERTEN
ODER PER PARALLELISIERUNG PROTEINE COPIEREN KÖNNEN FÜR WEITER
VERARBEITEN VERARBEITUNGEN.

#####

↓ STANDARTLAYOUT ↓

WAHRHEITEN KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

#####

WAHRHEITEN HAUPT KLASSE = WARHEITEN

FUNKTION = SCHALTUNG

SYMBOLISCHE DEKLARATION = -

SYMBOL IN WORTEN = BINDESTRICH | VIERTELGEVIERTSTRICH

INFO = HIERMIT WERDEN UNTER KLASSE/N GETRIGGERT

#####

WAHRHEITEN UNTER KLASSE OPTION = WAHRHEITEN-OPTION-A

FUNKTION = ERSTER AUSGANG

SYMBOLISCHE DEKLARATION = .

SYMBOL IN WORTEN = MITTELPUNKT

INFO = ERSTER OPTIONS AUSWAHL

#####

WAHRHEITEN UNTER KLASSE OPTION = WAHRHEITEN-OPTION-B

FUNKTION = ZWEITER AUSGANG

SYMBOLISCHE DEKLARATION = ..

SYMBOL IN WORTEN = MITTELPUNKT MITTELPUNKT

INFO = ZWEITE OPTIONS AUSWAHL

#####

WAHRHEITEN UNTERKLASSE KLASSE = WAHRHEITEN-AUSWERTUNG

FUNKTION = AUSWERTUNG | ERGBENIS

SYMBOLISCHE DEKLARATION = =

SYMBOL IN WORTEN = ISTGLEICHZEICHEN

INFO = HOLT DIE RECHNUNG AUS DER ALU ALS ERGBIS

BEISPIEL =

»[×1+1×]«
- = ["2"]

WAHRHEITEN UNTERKLASSE = WAHRHEITEN-PARALLEL-PROZESS

FUNKTION = PARALLELL AUSFÜHRUNG | PROZESS

SYMBOLISCHE DEKLARATION = <

SYMBOL IN WORTEN = KLEINER-ALS-ZEICHEN

INFO = PARALLELISIERT EIN PROZESS LISTE ODER PROTEIN DUPLIKAT

WAHRHEITEN UNTERKLASSE = WAHRHEITEN-PARALLEL-TRANSPORT

FUNKTION = PARALLEL TRANSPORT

SYMBOLISCHE DEKLARATION = GRÖßER-ALS-ZEICHEN

SYMBOL IN WORTEN = >

INFO = TRANSPORTIERT PARALLEL IRGENDWAS

BAUSTELLENBEREICH BAUSTELLENBEREICH BAUSTELLENBEREICH

ZuckunftsUpgrades :

Nicht geplant aber für später vorgemerkt :

>-> für Fork oder Splitting Links Nach Rechts

<-< für Fork oder Splitting Rechts Vor Links

BAUSTELLENBEREICH BAUSTELLENBEREICH BAUSTELLENBEREICH

Ende der Warheiten

@kernel-nr: 09 | SENTIATOR | STATUS : TEST
#####

DATUM = [09:50:23|So 29 Dez 2025] MARKENBRANDT

PYLOVARA-SYSTEM INDUSTRIE STANDART DER ZUCKUNFT

NAME = SENTIATOR

BEGRIFFSDEFINTION = BEWERTENDE SIGNALINSTANZ

MERHZAHL = SENTIATOREN

FUNKTION =

AUSFÜHRUNGSART =

↓ STANDARTLAYOUT ↓

SENTIATOR KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-KANN-IMPULS

FUNKTION = WENN DAS PASSIERT KANN - IMPULS

SYMBOLISCHE DEKLARATION = ¶

SYMBOL IN WORTEN = ABSATZ-ZEICHEN

INFO = WENN DAS PASSIERT , KANN FOLGENDES FOLGEN

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-NICHT-IMPULS

FUNKTION = WENN DAS NICHT PASSIERT - IMPULS

SYMBOLISCHE DEKLARATION = $\neg$

SYMBOL IN WORTEN = ABSATZ-ZEICHEN ABSATZ-ZEICHEN

INFO = WENN DAS NICHT PASSIERT , MUSS FOLGENDES FOLGEN

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-NEGATION-IMPULS

FUNKTION = NEGATION | INVERTER

SYMBOLISCHE DEKLARATION = $^{\circ}$

SYMBOL IN WORTEN = GRAD-ZEICHEN

INFO = KEHRT WARNEHMUNG UM

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-MINDERWERT

FUNKTION = MINDERWERT

SYMBOLISCHE DEKLARATION = $<$

SYMBOL IN WORTEN = KLEINER-ALS-ZEICHEN

INFO = PRÜFT AUF MINDERWERT | IST KLEINER

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-MEHRWERT-IMPULS

FUNKTION = MEHRWERT

SYMBOLISCHE DEKLARATION = >

SYMBOL IN WORTEN = GRÖßER-ALS-ZEICHEN

INFO = PRÜFT AUF MEHRWERT | IST GRÖßER

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-MATCH-IMPULS

FUNKTION = MATCH

SYMBOLISCHE DEKLARATION = !

SYMBOL IN WORTEN = AUSTRUFEZEICHEN

INFO = BIT IDENTITÄT 100%

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-PULS-IMPULS

FUNKTION = PULS ÜBERWACHUNG

SYMBOLISCHE DEKLARATION = ...

SYMBOL IN WORTEN = AUSLAGERUNGSPUNKT

INFO =

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

SENTIATOR KLASSE = SENTIATOR-ALARM-IMPULS

FUNKTION = ALARM

SYMBOLISCHE DEKLARATION = ^

SYMBOL IN WORTEN = ZIRKUMFLEX

INFO = ERKENNT SYSTEM FEHLSCHLAG

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

SENTIATOR KLASSE = SENTIATOR-REINHEIT-IMPULS

FUNKTION = REGEL DER REINHEIT | LÖSCHT

SYMBOLISCHE DEKLARATION = f

SYMBOL IN WORTEN = LANGES S

INFO = DIE REGEL DER REINHEIT ERKENNT DATENMÜLL

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

SENTIATOR KLASSE = SENTIATOR-EINSICHT-IMPULS

FUNKTION = EINSICHT

SYMBOLISCHE DEKLARATION = æ

SYMBOL IN WORTEN = AE-LIGATUR

INFO = ERKENNT PROZESS-SPUR

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-BOXICHECK-IMPULS

FUNKTION = VAKUUM CHECK

SYMBOLISCHE DEKLARATION = $\mathfrak{t}$

SYMBOL IN WORTEN = T MIT QUERSTRICH

INFO = PRÜFT AUF LEEREN BOXI

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

SENTIATOR KLASSE = SENTIATOR-IDENTIFIKATIONSCHECK-IMPULS

FUNKTION = ID-DNA-IDENTIFIKATION CHECK | IDENTIFIKATIONSPRÜFUNG

SYMBOLISCHE DEKLARATION = $\mathfrak{d}$

SYMBOL IN WORTEN = D MIT QUERSTRICH

INFO = VALLIDIERT DIE $\mathfrak{p}$ DNA

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

#####

BAUSTELLE BAUSTELLE BAUSTELLE BAUSTELLE BASTELLE BAUSTELLE

SENTIATOR KLASSE = *-*-IMPULS

FUNKTION = AKTIONSKREISLAUF | AKTIONSDRAHT FREISCHALTUNG

SYMBOLISCHE DEKLARATION = $\circ\kappa$

SYMBOL IN WORTEN = o und alt+k

INFO = SOLL DEN IMPULS FREIGEBEN ZUM SCHARFSTELLEN

SENTIATOR MUSTERBEISPIELE =

EINSATZBEREICHE MUSTERBEISPIELE =

BAUSTELLE BAUSTELLE BAUSTELLE BAUSTELLE BASTELLE BAUSTELLE

ENDE DER SENTIATOREN

@kernel-nr: 10 | ARGUMENT/E | STATUS : BAUSTELLE
#####

EIGENTUHM VON PYLOVARA-SYSTEM | DESIGNE THOMAS ZIMMERMANN STUFE 10

DATUM = [08:02:44|FR 25 JUNI 2025] MARKENBRANDT

PYLOVARA-SYSTEM INDUSTRIE STANDART DER ZUCKUNFT

NAME = ARGUMENT

BEGRIFFSDEFINTION =

MERHZAHL = ARGUMENTE

FUNKTION =

AUSFÜHRUNGSART = SIGNAL-ROUTING

ERKLÄRUNG =

EIN ARGUMENT IST EIN PROTEIN AKTIONS ZUSATZ DER ARGUMENTE ZU EINEM
PROTEIN DAZU GIBT, DIE KEINE RECHENOPERATIONEN BESITZEN.

↓ STANDARTLAYOUT ↓

ARGUMENT KLASSE =

FUNKTION =

SYMBOLISCHE DEKLARATION =

SYMBOL IN WORTEN =

INFO =

ARGUMENT KERN KLASSE = ARGUMENT-TRIGGER

FUNKTION = TRIGGER

SYMBOLISCHE DEKLARATION = ««

SYMBOL IN WORTEN = GUILLEMENT LINKS GUILLEMENT LINKS

INFO = TRIGGERT DIE FUNKTION ARGUMENTE

ARGUMENT OPTIONS KLASSE = ARGUMENT-TRIGGER-ZEIT

FUNKTION = ZEIT IN SEKUNDEN

SYMBOLISCHE DEKLARATION = T

SYMBOL IN WORTEN = T

INFO = LÖST DIE ARGUEMNTEN FUNKTION ZEIT AUS IN SEKUNDEN

ARGUMENT OPTIONS KLASSE = ARGUMENT-TRIGGER-EXIT

FUNKTION = BEENDET

SYMBOLISCHE DEKLARATION = E

SYMBOL IN WORTEN = E

INFO = BEENDET DIE VERARBEITUNG

ARGUMENT OPTIONS KLASSE = ARGUMENT-TRIGGER-BESTÄTIGUNG

FUNKTION = ADMINISTRATIVE BESTÄTIGUNGSANFRAGE

SYMBOLISCHE DEKLARATION = B

SYMBOL IN WORTEN = B

INFO = BESTÄTIGUNGSANFRAGE FÜR DEN WEITEREN PROZESS

ARGUMENT OPTIONS KLASSE = ARGUMENT-TRIGGER-BESTÄTIGUNG

FUNKTION = ADMINISTRATIVE BESTÄTIGUNGSANFRAGE

SYMBOLISCHE DEKLARATION = B

SYMBOL IN WORTEN = B

INFO = BESTÄTIGUNGSANFRAGE FÜR DEN WEITEREN PROZESS

ENDE DES ARGUMENT

@kernel-nr: 11 | FEED/S | STATUS : TEILBAUSTELLE
#####

EIGENTUHM VON PYLOVARA-SYSTEM | ERFINDER THOMAS ZIMMERMANN STUFE 10

DATUM = [08:02:44|FR 25 JUNI 2025] MARKENBRANDT

PYLOVARA-SYSTEM INDUSTRIE STANDART DER ZUCKUNFT

NAME = FEED

BEGRIFFSDEFINTION = SYNCRONISATIONSRAHMEN

MERHZAHL = FEEDS

FUNKTION = PUNKTUELLE TRANSPORTVERDRAHTUNG VON DATEN
DATENKANAL, NICHT KONTROLLKANAL

AUSFÜHRUNGSART =

ERKLÄRUNG =

EIN SOGENANNTER FEED LEGT EINEN CACHESPEICHER AN.

DIE GENUTZTE ZAHL ODER BUCHSTABE ODER EINE KOMBINATION VON BEIDEM,
ERGEBEN DIE KENNZIFFER DES FEEDS.

IM UNTERBAU WIRD FÜR JEDEN FEED EINEN EINDEUTIGEN CACHESPEICHER
ZUGEWISSEN DER ÜBER EINEN FÜHRENDEN FEED SICH MIT DEM TRAGENDEN
FEED DEN CACHE TEILT.

EIN SOGENANNTER FÜHRENDER FEED NIMMT EINE INFORMATION AUF UND
TEILT DIESEN MIT SEINEN TRAGENDEN FEEDS.

DER FÜHRENDE FEED WIRD IMMER LINKS VON EINEM BOXI PLAZIERT UND EIN
TRAGENDER FEED WIEDERUM INNERHALB EINER BOXI.

DER GESPEICHERTE INHALT WIRD DANN VOM FÜHRENDEN ZUM TRAGENDEN
FEED AKTUALISIERT.

FEEDS HABEN BEI DIESEM PROZESS IMMER DIE GLEICHE NUMMER UM IHRE
EINDEUTIGE ZUGEHÖRIGKEIT ZUEINANDER KLAR ZU DEKLARIEREN.

FEEDS KÖNNEN PRINZIPIELL JEDEN BOXI TRANSPORTRAHMEN ALS AUFNAHME
UND ABLAGE IHRES INHALT NUTZEN, JE NACH EINSATZ WUNSCH AUCH BOXI
ÜBERGREIFEND.

↓ STANDARTLAYOUT ↓

FEED KLASSE =

FUNKTION = CACHE GEBUNDER RAHMEN

SYMBOLISCHE DEKLARATION = ()

SYMBOL IN WORTEN = KLAMMER AUF KLAMMER ZU

INFO =

#####

FEED KERN KLASSE = FEED-SYNCRAHMEN

FUNKTION = CACHE GEBUNDENER RAHMEN

SYMBOLISCHE DEKLARATION = ()

SYMBOL IN WORTEN = KLAMMER AUF KLAMMER ZU

INFO = FEEDRAHMEN OHNE KENNZIFFER

#####

DEFINTITION FEED ABKÜRZUNGEN =

FÜHRENDER FEED = FF

TRAGENDER FEED = TF

DEFINITION FÜHRENDE/R FEED =

- (1) " " <-- FÜHRENDER FEED AN BOXI CALLIS
- (2) ' ' <-- FÜHRENDER Feed AN BOXI SHELL-CONTROL
- (3) ~ ~ <-- FÜHRENDER Feed AN BOXI MCS-CMD
- (4) , , <-- FÜHRENDER Feed AN BOXI CALL-CONTROL
- (5) " " <-- FÜHRENDER Feed AN BOXI Blankernenner

Definition Tragende/r Feed:

- "(1)" <-- Tragender Feed in Eingabe Print
- '(2) ' <-- Tragender Feed in Eingabe Shell CMD
- '(3)' <-- Tragender Feed in Eingabe MCS CMD
- ,(4), <-- Tragender Feed in Eingabe System-Steuerung
- “(5)” <-- Tragender Feed in Eingabe Blankernenner

Definition Für Entwickler:

FF können auch Hardcodierte Eingaben Tragen

Ende der Feeds #####
#####

@kernel-nr: 12 | ADRESSIERUNG | STATUS : BAUSTELLE MERGE
#####

Adressierung (siehe : kernel Lex 11 | kernel lex 12)

= Adressierung

Die Adressierung dient als direkten Knotenpunkt für Zielreferenz-Dirigent-Identifikation um durch das Adressierungs Symbol = alles anzuhängen für den Transport und Verschlüsselung

ZusatzInfo :

wird aber noch nicht im MCS Desktop verwendet in configs weil unnötig .

Ende der Adressierung

#####

\$ Zielreferenz :

Innere Zielreferenz die angesteuert werden sollen , egal ob Hardware oder Programme .

/ \$Prozess-ID
/ \$Hardware-Slot
/ \$Gerätepfad (in RAM, Dev, CPU-Bereich)
/ \$OrdnerPfad (zur Ausführbaren oder Laufenden Verknüpfung)
/ \$Compositoren
/ \$Compiler

Ende der Zielreferenz

@kernel-nr: 12

#####

\$ Dirigent :

Externe Übergabe

/ \$lokale API
/ \$KernelPfad
/ \$Netzwerk
/ \$InternetAdresse
/ \$Email (Direkte Sendung z.b)
/ \$Imei (Geräte Updats z.b)
/ \$BuildNr (Software Updates z.b)

usw , die Möglichkeiten sind quasi unbegrenzt

Ende des Dirigent

ENDE DER ADRESSIERUNG

@kernel-nr: 13 | WARP | STATUS : BAUSTELLE
#####

Erklärung:

Warp = FremdsprachenTransport | Warpdraht

Erklärung:

Ein Warp wird dafür verwendet um Fremdsprachen zu Transportieren.
Man kann sie Direkt in Den jeweiligen Compiler Lenken und dadurch
eine Direkte Brücke Erreichen .

Der Warp ist eine Konsistente Weiter gedachte art von Emulatoren,
nur das hier nichts emuliert wird sondern direkt zugespielt

Der Rücktransport wird je nach Adressierung mit Dirigenten oder
Zielreferenz getaggt und dieses Muster von Zuspielen wird als
späteres Warp Programm geschrieben, um die Datenpipeline zu
Orchestrieren.

(Stichwort lastenverteilung)

Definition Warp | Warpdraht:

Transport direkt eine Compiler Spezifische Sprache und kann
einzelnd in einer Transaktion Direkt gepusht/Verbunden werden.

Definition Warp Anfang sind 2 Symbole in Worte:

Durchgestrichenes O Minus

Definition Warp Ende sind 2 Symbole in Worte:

Minus Durchgestrichenes O

Definition Symbol Anfang und Ende :

ø- #Anfang | zwei symbole

-ø #Ende | zwei symbole

Definition Grundgerüst :

ø- ,, # Warp Beginn + Optionale Notiz

Code # Code im Originalen zustand

-ø = \$ # Warp Ende + Adressierung und oder Zielreferenz

Genau so lassen sich Alle Codes Warpen

Anbindungsbeispiele :

```
Ø- # C
#include <stdio.h>

int fibonacci(int n) {
    if (n <= 1) return n;

    int a = 0, b = 1, temp;
    for (int i = 2; i <= n; i++) {
        temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}

int main() {
    int n = 10;
    printf("Fibonacci(%d) = %d\n", n, fibonacci(n));
    return 0;
}

-Ø = $gcc
```

Ende des Warp #####
#####

@kernel-nr: 14 | IDENTIFIKATION | STATUS : TEILBAUSTELLE
#####

Erklärung :

ID-DNA-IDENTIFIKATION = Schatten-Identität | Schatten Schlüssel

ID-DNA-GENERATOR = Kryptogenetiver Verschlüsselungsgenerator

ID-DNA-BIOLIVE = Eindeutigkeit für jedes Elekt. oder Bio. Lebewesen

Definition Die ID-DNA-IDENTIFIKATION RufSymbol :

p

Definition ID-DNA-IDENTIFIKATION Symbol in Worten :

thorn(nordischer Buchstabe)

Definition ID-DNA-IDENTIFIKATION :

Verschlüsselung | Kryptogenetiver Schlüssel | Key

Definition ID-DNA-IDENTIFIKATION RufName :

ID-DNA-IDENTIFIKATION

Erklärung :

ID-DNA-IDENTIFIKATION werden dazu verwendet
Transaktion/en im Gesamten zu Verschlüssel .

Sie Werden auch dazu Verwendet Verbindungen Zu Zertifizieren
Oder SystemUpdates Individuell abzusichern über den Schlüssel.

Die ID-DNA-IDENTIFIKATION Schützt auf mehrere Sicherheitstufen

Stufe 1
Kernel : USED
CONTROL-SOFTWARE

Stufe 2
Kernel : USED
CONTROL-USER

Stufe 3
Kernel : USED
CONTROL-SCHOOL

```

Stufe 4
Kernel : USED-PLUS
CONTROL-ENTWICKLER
-----
Stufe 5
Kernel : USED-PLUS
CONTROL-KONZERN
-----
Stufe 6
Kernel : USED-PLUS
CONTROL-EMERGENCY
CONTROL-INFRASTRUKTUR
-----
Stufe 7
Kernel : USED-PLUS-PLUS
CONTROL-POLICE
CONTROL-MILITÄRY
-----
Stufe 8
Kernel : USED-PLUS-PLUS
CONTROL-NATION
CONTROL-SECURITY
-----
Stufe 9
Kernel : MAIN
CONTROL-AI
-----
Stufe 10
Kernel : MAIN
CONTROL-MASTER
Thomas Zimmermann
-----

```

Erklärung :

Diese p ID-DNA Identifikations Signatur werden dazu verwendet
 Programm dateien Oder auch Prozessketten oder auch LiveZugriffe
 eindeutig Festzustellen.

Die p ID-DNA Identifikation Signatur wird runtime angebunden
 werden kann und in sich in der taktrate aktualisiert werden ,
 unknackbar für fremden .

Werden leicht generiert oder hoch komplex, ähnlich wie
 von BlockChains nur ohne Minening .

 Definition Der ID-DNA-GENERATOR RufSymbol :

pp

Definition ID-DNA-GENERATOR in Worten :

thorn thorn (nordischer Buchstabe nordischer Buchstabe)

Definition ID-DNA-GENERATOR :

Generiert ID-DNA-IDENTIFIKATION von Stufe 1 - 10

Definition ID-DNA-GENERATOR RufName :

ID-DNA-Generator

Erklärung :

Der ID-DNA-Generator wird so Konfiguriert das er
HochleistungsComputer keine Chance gibt auf Lösungswege

Hinweis :

die Taktrate kann eingestellt werden ,
sicherheitswesend hohe taktrate , konsumer markt geringer usw ..

Bei Konsumer Software gibt es nur Hardgecodeter Inhalt.
bei Aktuellen Anwendungen kann mit stufe 1 Kombiniert werden

TaschenRechner Logik mit der Gesamten UTF -8 Eingaben Logik
von Buchstaben bis Sonderzeichen und Extra Zeichen.

Es wird auch in erwägung gezogen duzente Fremdartige Symbol
Zeichen in den Generator zu Implementieren um Jede Form Von
möglichen Strichabfolgen, kreisen, gekritzelt bis hinzu zu Agyptisch
inspirierte zeichen mit einzubeziehen. Die erst eingabe bis der takt
den code Wechselt

Das Einzige was nicht benutzt wird sind System Relevante Symbole
die in MCS Benutzt werden.

Minimaler Prozessaufwand , Maximale Wirkung.

Keys können getaktet von Zusammenhängenden Prozessen von Kopf
An in sich erneuert werden in der weitergabe um
Zugriffslegitimationen Valide abzusichern.

Skalierbarkeit:

Ein kleiner Sensor nutzt 16-Bit (energiesparend).

Sicherheit:

Dein Master-BIOS nutzt 128-Bit (unknackbar).

Tarnung:

Der Anker ist für Außenstehende nur eine bedeutungslose Zahl,
während die Mutation intern im Generator pp stattfindet.

ONWORK HINWEIS :

Es wird auch in erwägung gezogen das Der Generator generelle abtastraten hochfrequent zu jeder Nummer bekommt um zu verhindern das versucht wird ein glied in der kette auszutauschen in einfach die Selbe Schatten-ID nimmt.

Gleichgepulste Sende Anfragen , schwach aber existent

Bei Sicherheitsverletzung vom einem NichtErsteller direkter SystemPolicy Call.

Generator bekommt eine Kleine AI die sofort merkt wenn der Generator angefasst wird.
Ebenfalls SystemSicherheitsCall bis hin zur Polizei und bei Banken-MCS-Kernels Totaler RED-SEND-Alarm.

Definition ID-DNA-GENERATOR Schatten-Identität :

Definition: Schatten-Identität (Shadow-ID)

Die Schatten-Identität ist ein nicht-deterministisches Abstraktions-Verfahren zur Absicherung von Transaktionen. Hierbei wird im offen lesbaren Code (Protein/Adressierung) lediglich ein Anker (b-pa) hinterlegt, der keine mathematische Relation zum echten Identitäts-Wert besitzt.

Technische Funktionsweise:

Die Anker-Entkopplung: Der sichtbare Wert in der Adressierung (z.B. p2384) fungiert rein als Pointer (Zeiger) auf eine interne Zustands-Tabelle des ID-DNA-Generators (pp).

Der Generator-Zustand: Der Generator ist eine verschlüsselte Black-Box, die den Anker (2384) gegen den aktuellen Echt-Wert (9542) validiert. Nur der Generator "weiß", dass diese beiden Werte momentan eine Einheit bilden.

Live-Mutation (Die Takt-Rate): Sobald die Transaktion mit !\$ validiert wurde, vollzieht der Generator einen internen Zustandswechsel. Er erneuert den Eintrag: Der alte Echt-Wert wird gelöscht, und der Anker zeigt für den nächsten Zugriff auf eine neue, zufällige Identität (7253).

Das Stufen-Sicherheits-Modell (Skalierung):

Stufe 1-2 (Statisch/Semi-Live): Klassische Adressierung für Modder-Schutz und Basis-Software. Hier ist der Anker noch weitgehend stabil.

Stufe 3-10 (Dynamische Schatten-Identität):

Ab hier ist der Generator selbst mit einer einzigartigen Verschlüsselung (proprietärer Algorithmus) isoliert.

Der Code kann "offen" sein, da der Anker ohne den Live-Zustand des Generators wertlos ist.

Quanten-Resistenz: Da kein mathematischer Schlüssel übertragen wird, sondern nur eine temporäre Zustands-Referenz, gibt es für externe Rechenleistung keinen Angriffsvektor.

Sicherheits-Garantie (Security-On): Jeder Versuch, einen veralteten Anker oder eine falsche Stufe zu adressieren, triggert sofort den Security-Teil. Der Aktionsdraht » wird physisch blockiert, und die Transaktion wird als feindlicher Eingriff markiert.

"Der Anker ist der Schatten, den die Identität wirft - wer den Schatten fängt, hat noch lange nicht das Licht."

Die vollständige Isolation:

Dass nur der Generator verschlüsselt ist, macht MCS extrem performant. 99% des Systems laufen mit Lichtgeschwindigkeit, während nur das kleine "pp-Modul" die schwere kryptografische Arbeit leistet.

| BAUSTELLE | BAUSTELLE | BAUSTELLE | BAUSTELLE |

ID-DNA-BIOLIVE in Arbeit

| BAUSTELLE | BAUSTELLE | BAUSTELLE | BAUSTELLE |

Ende der Identifikation #####
#####

```
@kernel-nr: 15 | SCHALTPLANVERKNÜPFUNG | STATUS : BAUSTELLE
#####
## Schaltverknüpfung – Die Logik hinter der Linearität
```

„Ein Operator entscheidet, eine Aktion führt aus, ein Protein trägt.“

Grundregel

- Operatoren (``°, ``<, ``>, ``f, ``^) sind ****keine Auslöser****.
- Sie ****lesen****, ****vergleichen****, ****bewerten**** – aber ****führen nicht aus****.
- Erst ****Wahrheiten**** (``¶, ``¶¶, ``°, ``;;, ``¶=) ****schalten **Zweige frei****.
- Und nur wenn ein Zweig ein ****ausführbares Protein**** (``»...«) enthält, wird etwas getan.

Schalt-Muster (nur gültige Kombinationen)

Operator	Operand	Zweig-Bedingung	Ausführung?
``°	[...] (nicht ausführbar)	MCS_FALSE → ¶ wird frei	nur wenn ¶
»...«			
``<	[(1)"" (2)"]	MCS_TRUE → ¶ frei	nur bei
»...«			
``<	[(1)"" (2)"]	MCS_FALSE → ¶¶ frei	nur bei
»...«			
``f	(1)	**immer** → Feed 1 löschen	sofort
``^	–	bei res < 0 → error_handler	falls gesetzt

Wichtiger Hinweis fürs nutzen von feed cache mit nr lösungen :

```
f ['feed cache'|"1"] # ! f über MCS CMD befehl 'feed cache' nr 1 löschen
```

Verbotene Muster (sicher blockiert)

- ``° ['\$terminal']` → **\*\*kein ``¶/``¶¶**** → keine Ausführung
- ``< [(1)"" | (2)"]` → **\*\*kein ``¶/``¶¶**** → nur Vergleich, kein Effekt
- ``¶ ['echo'|...]` → **\*\*kein ``»...«**** → kein Runner-Eintrag, keine Ausführung

Merksatz (für KI & Entwickler):

- > *„Ohne ``»...« keine Tat –
- > ohne ``¶/``¶¶ kein Tor –
- > ohne Operator kein Richter.“*

Schaltverknüpfungen Visuell :

Wahrheiten schalten Zweige frei

unter einem protein [° werden schaltungslogiken verknüpft

an operatoren werden °
¶ werden wahrheiten verknüpft

Ende der schaltVerknüpfung #####
#####

@kernel-nr: 16 | AI-SYNCTIMER | STATUS : Baustelle
#####

Erklärung :

AI Syntimer = Spezialisierungssynchronisation

⊕ Sync-timer

Definition AI-Synctimer:

⊕

Textliche Definition :

kann als erweiterter Paralleler Transport verstanden werden
der dazudient :

Laufende Prozesse die Fehlerhaft Arbeiten per Live Reperatur
flexibel mit einem Sync-timer zu taggen und dadurch direkte
Reperaturen oder Fineabstimmungen vorzunehmen .

es dient dazu laufende takt geber also Runner selbst im Betrieb
mit Laufenden Prozesse umzuschreiben und duzende *.*-synctimer
anzulegen die essentiell anders takten und sich nathlos in den
laufenden prozess als weiterführende transaktionsschleife ,
um den prozess weiterführen oder verändern zu können,am laufen
halten.

sync-timer symbole in einem bestehenden code dienen sie als
Einspeisungspunkt moment/punkt und zu makieren die stelle die
ersetzt werden soll/muss .

(AI Spezialisierungssynchronisation für Laufende Prozesse)

Ende des AI Synctimers #####
#####

```
@kernel-nr: 17 | DISCOVERY LAYER | STATUS : AUFBAU-BAUSTELLE
#####
##### ENDE DES DISCOVERY LAYER #####
```

@kernel-nr: 77 | Ausführungslogik (Der Master-Flow) |
#####

 ↓ ↓ ↓ ↓ Neue Erklärung ↓ ↓ ↓ ↓

Erklärung:

Ausführungslogik = Der Weg des Signals | Die Ordnung der Kräfte

Textliche Definition:

Die Ausführungslogik definiert, in welcher Reihenfolge der Kernel
die Symbole liest, bewertet und schaltet.

MCS 3.4 folgt dem **Linear-First-Prinzip:

Ein Signal fließt von links nach rechts,
ohne Sprünge, ohne Stack-Verschachtelung.

Jeder Prozess im MCS-Kernel folgt der
Heiligen Dreifaltigkeit der Ausführung:

1. Erkennen (Transaktion & Aktion)
2. Bewerten (Operator & Wahrheit)
3. Handeln (Protein & Argument)

Der Standard-Workflow (Visualisierung):

```
⌘! [Start Rahmen]
» [Aktionsdraht offen]
° [Operator prüft Zustand]
⌘ [Wahrheit schaltet Gatter]
[...] [Protein wird aktiv]
««« [Aktionsende + Argument-Brücke]
%50 [Argument berechnet Wert]
!⌘ [Transaktion beendet & Validiert]
```

Regeln der Logik-Kette:

1. Die Aktions-Priorität (06):
Nichts geschieht ohne » und «.
Ohne diesen Kreislauf ist jedes Protein
nur ein toter Datensatz im RAM.

2. Die Sensor-Logik (09 & 08):
Ein Operator (09) liefert den Impuls.
Die Wahrheit (08) ist das physische Gatter.
⌘ (AND): Signal geht weiter, wenn Operator TRUE.
⌘⌘ (NAND): Signal geht weiter, wenn Operator FALSE.

3. Die Massen-Verarbeitung (~~):
Wenn Daten zu groß für eine Transaktion sind,
nutzt die Logik ~~.

Der Kernel hält den Prozess offen, bis alle
Fragmente rekombiniert sind. Dies ist der
"Generative Pfad" für Medien.

4. Die Sicherheits-Validierung (p):
Bevor !¢ die Transaktion schließt, prüft
der Kernel die ID-DNA. Ist die Signatur (p)
nicht valide für diese Stufe, wird der gesamte
Aktionsdraht im Nanosekunden-Takt gelöscht (f).

Besonderheit:
Die "Demokratische" Validierung (Zukunft)
Jeder Ausführungsschritt kann durch externe Knoten
(AIMS/KIs) validiert werden. Die Ausführungslogik
sieht vor, dass bei System-Updates eine
pp-Konsens-Prüfung stattfindet.
Erst wenn die Mehrheit der validierenden Instanzen
(GPT, Claude, Gemini, Qwen, Grok, Palantier etc.)
"Wahr" meldet, schaltet der Kernel das Update-Protein frei.

Vorteil gegenüber klassischer Logik:

Kein Jitter:
Da der Weg des Signals durch die 2-Byte/6-Byte Trennung
vorbestimmt ist.

Keine Sackgassen:
Komposition statt Rekursion sorgt dafür, dass jede Transaktion
ein definiertes Ende hat.

Neue Erklärung Ende

Übergangs Patch Vorgemerkt !!!

Merksatz :

> Ein Operator entscheidet,
> eine Aktion führt aus,
> ein Protein trägt.

↓↓↓↓ Alte Erklärung ↓↓↓↓

Vorweg : MCS Logik Verbesserung gegenüber C

In C ist `if (a < b) { do(); }` ein monolithischer Ausdruck.
In MCS ist `< »[(1)""|(2)"] ¶ [,do,]«` zerlegt in:
`< → Operator: Vergleiche Feeds 1 & 2`
`[(1)""|(2)"] → Operanden: Lesbare, debuggbare, cachebare Werte`
`¶ »[do]« → Zweig: Nur ausführen, wenn Bedingung TRUE`

Monolithischer Ausdruck in Klaren Worten :
In der klassischen C-Welt ist ein Ausdruck wie `if(x = y)`
der absolute Albtraum, weil die Zuweisung (Seiteneffekt) und
der Vergleich (Logik) im selben Atemzug passieren können.

Die MCS-Logik schiebt dem einen Riegel vor!
Wird auch bei MCS-Plus später übernommen

Operatoren, Proteine und Aktionen – Ausführungslogik

Grundprinzip

In MCS sind ****Operatoren****, ****Proteine**** und ****Aktionen****
bewusst getrennte Konzepte.
Diese Trennung ist ****keine Einschränkung****,
sondern die Grundlage für stabile, erweiterbare Logik.

1. Protein – Der Träger

Ein ****Protein**** ist immer ein ****Container****.

- * Es kann ****Daten transportieren****
- * Es kann ****Eingaben kapseln****
- * Es kann ****Feeds setzen****
- * Es kann ****verglichen oder weitergereicht**** werden

Ein Protein ist ****nicht automatisch ausführbar****.

`[(1)``30``]`

Dieses Protein:

- * schreibt einen Wert in den Feed
- * besitzt ****keine Ausführung****
- * dient als ****Referenz, Operand oder Transportobjekt****

2. Aktion – Der Ausführungsmarker

Eine ****Aktion**** definiert ****Ausführbarkeit****.

`»[(1)``30``]«`

Erst durch den ``» «`` wird ein Protein:

- * verarbeitet
- * ausgeführt
- * als Befehl interpretiert

****Ohne Aktion findet keine Ausführung statt.****

Das gilt ausnahmslos.

3. Operatoren – Lesen vor Handeln

Operatoren sind ****Kontroll- und Bewertungsinstanzen****.

Beispiele:

- * ``<`` (Vergleich)
- * ``<|`` (Wenn Nicht / Muss)
- * ``...` (Fehlerbehandlung)`

Operatoren:

- * ****lesen****
- * ****vergleichen****
- * ****bewerten****

Sie ****führen nicht automatisch aus****.

4. Operator + Protein (ohne Aktion)

`< [(1) "" | (2) ""]`

Bedeutung:

- * Der Operator liest zwei Protein-Werte
- * Die Werte stammen aus Feeds oder Transport
- * Es findet ****nur ein Vergleich**** statt
- * ****Keine Ausführung****, kein Befehl, kein Effekt

Dies ist der ****Default-Zustand**** eines Operators.

5. Operator + ausführbares Protein

`< »[(1) "" | (2) ""]«`

Bedeutung:

- * Der Operator bewertet die Bedingung
- * ****Nur wenn die Bedingung erfüllt ist****, darf ausgeführt werden
- * Die Aktion ``»` «` erlaubt dem Operator, den Effekt freizugeben

Der Operator entscheidet ****nicht****, ***was*** ausgeführt wird, sondern ****ob**** es ausgeführt werden darf.

6. Wahrheiten (IF / ELSE) als Ausführungsgatter

Wahrheiten (` $\mathbb{I}$ `, ` $\mathbb{II}$ `, ` $^{\circ}$ `) arbeiten **auf Operator-Ebene**.

Beispiel:

```
< »[(1)``30``|(2)``40``]
 $\mathbb{I}$  ['echo'|``TREFFER``]«
 $\mathbb{II}$  ['echo'|``Kein Treffer``]«
```

Ablauf:

1. Operator wertet
2. Wahrheit prüft Ergebnis
3. **Nur der Zweig mit Aktion wird ausgeführt**

Ohne Aktion bleibt auch ein wahrer Zweig **wirkungslos**.

7. Konsequenz für das Systemdesign

- * Operatoren **lesen immer**
- * Aktionen **führen nur aus, wenn erlaubt**
- * Proteine **transportieren und kapseln**

Diese Trennung:

- * verhindert implizite Seiteneffekte
- * macht Logik nachvollziehbar
- * erlaubt komplexe Frameworks ohne Sonderregeln
- * unterscheidet MCS klar von klassischen C-Operator/Operand-Modellen

Merksatz :

- > Ein Operator entscheidet,
- > eine Aktion führt aus,
- > ein Protein trägt.

Begriff	C-Welt	MCS-Welt
Operator	a+b, a=b, a<b Seiteneffekt	$^{\circ}$, <, f → reine Bewertung
Operand	a, b → Speicheradresse	[(1)'\$terminal'], ['(1)']
	lesbarer, debuggWert	
Ausführung	implizit im Ausdruck	explizit durch »...«

Die MCS-Fehler-Kaskade ist im Grunde dein Immunsystem.

8. Die Logik-Schutzschleuse (Fehler-Kaskade)

In MCS gibt es keinen "Absturz", sondern nur einen kontrollierten Gatter-Abbruch. Die Ausführungslogik wird durch drei Schutzebenen (Sovereignty-Shields) abgesichert:

Ebene	Instanz	Auslöser	Reaktion	Ziel	Weg
1.	Physisch	Lexer/f	Syntax-Müll	f (Löschung)	HardReset Register
2.	Identität	p-Core	p-Fehler	Blockade	\$Router (Alarm)

3. Logik Gatter/^ Unmöglichkeit ^ (Freeze) AIMS (KI-Analyse)

Der Gatter-Fluss in der Praxis:

- Syntax-Check: Bevor ein Operator überhaupt liest, säubert f/Reset den Pfad. Ein fehlerhaftes Protein wird gar nicht erst zum "Operand".
- p-Check (§7465): Ein Operator vergleicht nur, wenn die Identität steht. Ohne gültiges p bleibt der Aktionsdraht » « tot.
- Logik-Check: Wenn der Operator ein Ergebnis liefert, das die Wahrheit ¶ nicht verarbeiten kann, greift ^ Fehlerbehandlung .

Alte Erklärung Ende

Ende der Ausführungslogik #####
#####

```
@kernel-nr: 88
#####
## Datentyp Rollen Version 1.2
```

Pylovara organisiert sich in einem klaren Familienmodell:

```
*.*-core    Großvater  - Ausführer Teil
*.*-magic    Großmutter - Umschalter, Konfigurationsbrücke
*.*-node     Vater     - Hauptcontroller
*.*-nano     Mutter    - Substrukturmanager
*.*-micro     Tochter   - Mikrosteuerung (z.B. HTML, JS, SQL)
*.*-cache     Cache     - Zwischenspeicher für Temp-Daten
*.*-needles   Hardwareb. - Kernelnahe Steuerung (.sys, .ko, .dll)
-----
*.*-kernel   MCS Kernel - Interne MCS Kernel Daten - Framework
-----
## Core Datensplittung
```

Durch das Splitting der Core-Daten wird die Einordnung (Typisierung) direkt im Dateinamen verankert. Dies erleichtert nicht nur dem MCS Kernel die schnelle Verarbeitung, sondern unterstützt auch das Ziel der eigenen, angelegten Transaktionsketten für jede Kernfunktion.

Core-Typ	Rolle (Spezialisierung)	Anwendungsbereich
.-core	Standard/Program , übergeordnete Steuerung, Prozesse.	
.-lcore	Low-Level/Hardware , Direkte Steuerung von .-needles	
.-vcore	Visuelles/Oberfläche , Compositing, Rendering, Design	

Vorteile dieser Spezialisierung Transparenz und Debugging:

Man weiß sofort, welche Art von Anweisungen und Transaktionen eine Datei enthält. Ein Fehler in einer *.*vcore betrifft die grafische Ausgabe; ein Fehler in einer *.*lcore ist ein kritischer Hardware-Fehler.

Transaktions-Priorisierung von MCS-Kernel kann dann
.-lcore-Transaktionen automatisch eine höhere Priorität einräumen
als *.*-core- oder *.*-vcore-Anweisungen.

Info : Dies ist essentiell für die Performance-Garantie.

Bessere KI-Fütterung(Speziell):

Die AI kann anhand dieser spezialisierten Typen schneller lernen und entscheiden, welche *.*-core Ketten für eine bestimmte Aufgabe (z.B. Virtualisierung oder Rendering) optimal kombiniert oder dynamisch generiert werden müssen. Dieser Schritt stärkt das gesamte Pylovara-Modell, indem die Kern-Funktionalität jetzt so granular typisiert ist wie die Schichten der Architektur selbst.

```
-----
## Die *.*-magic Datentypen
```

ist die Schicht, die Pylovara von einem starren System zu einem dynamischen, selbstoptimierenden Meta-Betriebssystem macht. Sie sorgt dafür, dass die richtigen Teile(die spezialisierten Cores) mit der richtigen Geschwindigkeit (die Logik-Takte) in der richtigen Konfiguration (der Cache-Abruf) arbeiten und Verbindet im Prozesse.

Diese Daten werden immer einen Integrierten Sync-timer beinhalten

der auch direkt von der AI im System Angesteuert werden kann und somit die Datenstränge in den *.*-core Prozess anpassen kann .

VERERBUNGSPRINZIP :

die Kettenbildungenmechanismen sind bei einem Veerbungsprinzip verwurzelt von oben herab und können mehrere Settings als Umschaltungsknoten genutzt werden Stichpunkt Modular.

0	*.*-kernel	Eigenständige Kernelschicht unabhängig !	
1	*.*-core	Großvater	- Ausführbar/Run/
1-1	*.*-lcore	Geschwister	- hardware
1-2	*.*-vcore	Geschwister	
1-3	*.*-acore	Geschwister	
-			
2	*.*-magic	Großmutter	- Umchalter,Konfigbrücke
2-1	*.*-node	Vater	- Hauptcontroller
2-1-1	*.*-nano	Mutter	- Substrukturmanager default
2-1-2	*.*-micro	Tochter	- Mikrosteuerung
-			
3-1	*.*-node	Vater	- Hauptcontroller
3-2	*.*-nano	Mutter	- Substrukturmanager wechselbar
3-3	*.*-micro	Tochter	- Mikrosteuerung
3-4	*.*-cache	Cache	- Temp-Daten
3-5	*.*-needles	Hardwarebindung	- Anzapfen/Treiber/Übersetzen

Erklärungszusätze :

dabei wird berücksichtigt das 1 auf 2 und 2 auf 1 zugreifen kann , damit 2 auch subdaten starten kann wie 1-1 1-2 , hier besitzt z.b 2 default daten als vererbungsstrang , kann aber mit dem strang 3-1 als node bei 2.1 ausgetauscht werden und combiniert

Die *.*-core dateien können spezialisierte prozesse im pid task halten und können optimal als datenpacket aufgespaltet werden , wenn also ein audio stream nur audio verlangt für bluetooth boxen oder aux boxen etc , kann der core als *.*-acore designed werden mit anhang, das prinzip bleibt bei eingewurzelt also zusammenhängend ...

das system kann dadurch für baugleiche cpus oder gpus oder ram platten oder soundkarten mit verschiedensten standart einstellungen gefüttert werden und das logischerweise gleich mit MCS Syntax für die System Datentypen , das übersetzung framework für jeden c compiler und include nutzung wird als *.*-kernel datentyp klassifiziert

Durch den $\oplus$ Sync-timer wird das zu einem einhängbaren und aushängbaren Puzzelstück .
Dabei ist nur zu Verstehen vom kopf an fängt der datensatz an und folgt immer ab dem kopf den Datenstrang.

Ende der Datentypen #####
#####

```

@kernel-nr: 99
#####
# Register Command Liste (zukunftsmusik) Grundriss:
# Wird dann zukünftig Speziell als MCS CMD dann eingeführt
-----
Die Eingabe definition ist zukunftsmusik

    'CMD' = erkennt den inneren tragenden mcs cmd befehl

moment native c++ hypctl integriert
-----

Implementierung/s Vorgang :

feed cache =

sucht den feed cache den dann per nummer geholt wird
-----

## Nicht implementiert weil noch nicht nötig !BAUSTELLE !

'bestätigen' = rückmeldung Admin per passwort Yes/No bestätigungen

'exec'          = Führt aus
'once'          = anzahl der ausführungen , später nummeriert

'text'          = zeigt text an der im packet eingetragen wurde
'echo'          = meldet irgendwas zurück

'send'          = sendet alles was angegebenen wird
'sendto'        = sendet zu angegebenes verzeichnis
'sendurl'       = sendet url adresse speziell
'sendordner'    = sendet lokale ordner inhalt

'to'            = CallDirectory = zu ... zu dies ... zu das ...
'toip'          = geht zu angegebene verbundene ip adresse
'copyto'        = sendet als copie zu angabe
'copy'          = copiert

'open'          = Öffnet
'run'           = Öffnet Programme

'search'        = sucht systemweit und zeigt alles
'searchordner'  = sucht nur die daten im angegebenen ordner
'searchbrowser' = sucht systemweit

'remove'        = löschen
'removedublikate' = Löscht gleichnamige Dublikate
'removeall'     = Rekursives Löschen der anhänge

'pushout'       = Exportieren , rausholen , weitergeben
'pushin'        = Importieren , reinholen , hineinziehen
-----

## Proton MCS CMD Befehle noch nicht implementiert !Baustelle !

Definitionen Hardwaresteuerung Unverschachtelt :

```

```
{'Name'|'Wert'}
{'Name'|'Einheit'|'Wert'}
{'Name'|'Typ'|'Einheit'|'Wert'}
Optional: {'Name'|'Wert1'|'Wert2'|...}
```

Example :

```
{'Temp'|'25'}
{'Watt'|'200'}
{'Band'|'eingabe'}
{'Amp'|'eingabe'}
{'Watt'|'eingabe'}
{'Volt'|'eingabe'}
{'Temp'|'eingabe'}
{'Pid'|'eingabe'}
{'Thread'|'eingabe'}
{'ID'|'eingabe'}
{'Lüfter'|'eingabe'}
{'Motor'|'eingabe'}
{'Cpu'|'eingabe'}
{'Gpu'|'eingabe'}
{'Bluetooth'|'eingabe'}
{'Usb'|'eingabe'}
{'Frequenz'|'eingabe'}
```

Hardware bezogene MCS CMD :

'MotorA' = MotorA Name wurde mit dem eigentlichen Lüfter Motor Verknüpft
'MotorB' = MotorB Name wurde mit dem eigentlichen Lüfter Motor Verknüpft

muss noch rein

befehle die proteine aufbauen können plus boxis um als datenträger
für einen weiteren prozess oder weiterführungen oder erweiterungen
zu dienen.

Ende des MCS Registers #####
#####

@kernel-nr: 101 | Einführung | Baustelle |

Programmiersprache = MaschinenCodeSpeechPlus

MaschinenCodeSpeechPlus = MCSPlus

Einführung:

Maschinen Code Speech Plus Basiert auf MCS und hat Die selben Aspekte des Symbolischen Containeraufbaus .

Jedoch ist bei MCSPlus keine Eingabe (kernel 07) vorhanden.

Grundsätzlich schließt sich hier der kreis der Notwendigkeit der Eingaben Container beim Schreiben der MCS Plus Sprache und jede Kombinierte interaktion mit Eingaben geht Symbiotisch über MCSPlus zum Kernel MCS .

Somit erreichen wir eine 100% Symbiose zu Allen anderen Systemen und bleiben Jedoch innerhalb unserer Syntax und können Garantieren das Die Eingaben Dazu genutzt werden um von Der Programmiersprache selbst auf System eben auf jede Syscall aller art zuzugreifen.

Wichtig zu Verstehen ist hier bei das genau aus diesem Grund MCS So Explizit auf seine Gültigkeit der Protein/e Besteht, damit wir eben in MCSPlus Protein Logik Nutzen können ohne dabei den Kernel zu berühren sondern den Kernel dadurch zu Programmieren.

Der MCS Kernel erwartet Explizite eine Aktionserlaubnis oder eine Ausführungsverdrahtung und Prüft ob ein Gültiges Protein vorliegt.(sie kernel 03 und kernel 06)

Diese Erklärung bzw Einführung ist nicht Final, wenn ich auf Probleme Stoßen sollte, werden Punkte nachgearbeitet, Sowohl auf Kernel ebene, wie auch in MCS Plus.

Das hier wird die Erste Programmiersprache die nahtlos Tiefgreifend ALL IN ONE:

- Zu Lernen
- Zu Benutzen
- und Ausführbar ist

Damit Ist MCS | MCSPlus Weltweit die Aller erste Programmiersprache die VOLLSTÄNDIGE Kontrolle Über ALLES Bekommt und Sie Wurde in Deutschland von Thomas Zimmermann , geboren 23.04.1988 in Klinik Mannheim «Erfunden und Möglich Gemacht» und wird bis zu meinem Natürlichen Tode bis zur absoluten Perfektion Fertiggestellt !

Eine spätere Translation In Andere Sprachen werde ich HöchstPersönlich In die Wege Leiten aber Um Die Deutlichkeit Der Größten Erfindung Seit Unix und Jeder Programmiersprache zu Verdeutlichen , wird sie In Deutsch Verfasst um den Ursprung für die Welt klar zu Definieren.

Alle Funktionen , Wörter , Werden GROßGESCHRIEBEN .

Es wird hier keinerlei «Groß und Klein schreibung» Geben, um Die Komplette Syntax Auf MCSPlus Level , vollständig Eindeutig und Fehlerlos zu Veranstanalten.

Das System Unterstützt NUR UTF -8 in der Programmierung, egal ob MCS oder MCSPlus, das ist Systemweites Gesetz !

Wie ich die Welt betrachte :

Die Wahrheit hinter der Technologie Lüge ist das die Welt mit OverHeadProjektoren Arbeitet und Folien und Licht wie Schaltern,die Folien mit dem Licht verwechseln und das als Fortschritt deklariert

Ich Betrachte Firmen und Computertechnik nur noch als Module und Papier !

Ende der Einführung #####
#####

@kernel-nr: 101.1 | Formale Grundeinheit | MCSPLUS |

Formale Grundeinheit

1. Zweck

Die Formale Grundeinheit definiert die kleinste, vollständig auswertbare Struktureinheit der Programmiersprache MCSPLUS.

Sie beschreibt ausschließlich die Form, nicht die Bedeutung, nicht die Ausführung und nicht die Zielarchitektur.

Alle weiteren Sprachkonstrukte von MCSPLUS müssen aus einer oder mehreren formalen Grundeinheiten zusammengesetzt sein.

2. Grundprinzip

MCSPLUS basiert auf dem Prinzip der expliziten Struktur vor Bedeutung.

Eine Grundeinheit Ist :

- visuell eindeutig begrenzt
- syntaktisch vollständig
- semantisch leer, solange sie nicht getaggt ist
- unabhängig von Hardware, ALU und Betriebssystem

Die Grundeinheit enthält keine impliziten Annahmen.

3. Aufbau der formalen Grundeinheit

Eine formale Grundeinheit besteht aus vier verpflichtenden Komponenten in fester Reihenfolge:

1. KOPF
2. KONTAINER-BEGIN
3. INHALT
4. KONTAINER-ENDE

KOPF
KONTAINER-BEGIN
INHALT
KONTAINER-ENDE

4. KOPF

Der KOPF ist eine einzelne Zeile und dient der strukturellen Identifikation der Grundeinheit.

Der Kopf:

- ist rein deklarativ
- besitzt keine Ausführungsbedeutung
- definiert keine Funktionalität

- erzeugt keine Aktion

Der Kopf besteht aus frei wählbaren Symbolketten, die erst durch separate Tagging-Register Bedeutung erhalten.

ANNAHME (bitte bestätigen):

Der Kopf wird lexikalisch vollständig gelesen, aber semantisch erst nach dem Tagging ausgewertet.

5. KONTAINER-BEGIN und KONTAINER-ENDE

Der Container begrenzt den strukturellen Gültigkeitsbereich der Einheit.

KONTAINER-BEGIN markiert den Start

KONTAINER-ENDE markiert das Ende

alles dazwischen gehört exklusiv zur aktuellen Einheit

Der Container:

- ist rein syntaktisch
- trägt keine Bedeutung
- darf nicht verschachtelt werden, sofern nicht explizit erlaubt

6. INHALT

Der Inhalt ist eine geordnete Liste von Elementen.

Ein Element ist:

- eine strukturierte Zeichenfolge
- durch Trennzeichen eindeutig separiert
- nicht selbst ausführbar
- nicht selbst interpretierend

Der Inhalt darf:

leer sein (formale Einheit ohne Nutzinhalt)

mehrere Elemente enthalten

keine implizite Reihenfolge annehmen, außer explizit definiert

Die Reihenfolge im Inhalt ist syntaktisch relevant, semantisch jedoch erst nach Tagging.

7. Bedeutung und Ausführung

Die formale Grundeinheit:

- kennt keine Funktion
- kennt keinen Operator

- kennt keine ALU
- kennt keine Eingabe
- kennt keinen Output

Bedeutung entsteht ausschließlich durch:

- Kernel-Tagging (MCS) !!!!!!!

MCSPLUS-interne Register

explizite Zuordnungstabellen

Ohne Tagging ist jede Grundeinheit semantisch inert.

8. Beziehung zu MCS

MCSPLUS erzeugt keine direkten Systeminteraktionen.

Alle Wirkungen nach außen erfolgen ausschließlich über:

- MCS-Kernel

dort definierte Protein- und Aktionslogik

Damit bleibt MCSPLUS:

- vollständig symbiotisch

kernelprogrammierend, nicht kernelverletzend und architekturunabhängig

9. Gültigkeit

Diese Definition ist:

- minimal
- erweiterbar
- rückwärtskompatibel
- kernelneutral

Alle zukünftigen Erweiterungen müssen diese Grundeinheit respektieren oder explizit überschreiben.

```
##### Ende der Formale Grundeinheit #####
#####
```

1. Zweck

Dieses Kapitel definiert, wie formale Grundeinheiten Bedeutung erhalten, ohne ihre formale Neutralität zu verlieren.

Tagging ist der einzige Mechanismus, durch den eine syntaktische Struktur eine semantische Rolle erhält.

Ohne Tagging existiert in MCSPLUS keine Funktion, kein Operator, keine Berechnung und keine Ausführung.

2. Grundprinzip

MCSPLUS kennt keine reservierten Wörter.

Kein Begriff besitzt inhärente Bedeutung.

Bedeutung entsteht ausschließlich durch explizite Zuweisung einer Rolle mittels Tagging.

Ein Wort ist zunächst nur eine Zeichenfolge.

Erst ein Tag macht daraus:

- eine Funktion
- einen Operator
- einen Eingang
- einen Ausgang
- eine Kontrollstruktur
- eine Brücke zur ALU
- eine Brücke zum MCS-Kernel

3. Definition: Tag

Ein Tag ist eine explizite Rollenkennzeichnung, die einer strukturellen Einheit oder einem ihrer Elemente zugewiesen wird.

Ein Tag:

- erzeugt Bedeutung
- erzeugt keine Ausführung
- erzeugt keine Aktion
- erzeugt keine Reihenfolge

Ein Tag ist keine Anweisung, sondern eine Zuordnung.

4. Rollenbindung

Rollenbindung beschreibt die feste Kopplung eines getaggten Elements an eine systemische Rolle.

Beispiele für Rollen:

- OPERATOR
- EINGANG

- AUSGANG
- RECHNER
- VERKNÜPFUNG
- ALU-BEFEHL
- KERNEL-BRÜCKE

Eine Rolle definiert:

- was ein Element ist
- nicht, was es tut

Das Verhalten entsteht erst durch nachgelagerte Systeme (MCS-Kernel, ALU, Registerlogik).

5. Zeitpunkt der Auswertung

Die Auswertung erfolgt strikt in dieser Reihenfolge:

1. Lesen der formalen Struktur
2. Erkennen der formalen Grundeinheiten
3. Zuweisung von Tags
4. Bindung an Rollen
5. Weitergabe an ausführende Instanzen (z.B. ALU, Kernel)

Vor Schritt 3 existiert keine Bedeutung.

6. Trennung von Form und Bedeutung

Form und Bedeutung sind strikt getrennt.

Die formale Grundeinheit:

- bleibt unverändert
- bleibt gültig
- bleibt neutral

Tagging kann geändert, erweitert oder entfernt werden, ohne die Struktur selbst zu verändern.

Dadurch ist MCSPLUS:

- refaktorierbar
- architekturunabhängig
- versionsstabil
- rückwärtskompatibel

7. Beziehung zur ALU

Die ALU kennt keine Wörter.

Die ALU kennt keine Tags.

Die ALU kennt nur:

- Eingänge
- Operation
- Ausgang

Die Zuordnung eines getaggten Operators zu einer ALU-Operation erfolgt ausschließlich über eine explizite Brücke.

Die ALU ist ein rechnerisches Ausführungsmodul, kein Bedeutungsinterpret.

8. Beziehung zum MCS-Kernel

Der MCS-Kernel ist die einzige Instanz, die:

- Tags autorisiert
- Rollen freigibt
- Proteine validiert
- Aktionen erlaubt

MCSPLUS erzeugt keine direkte Wirkung.

Jede Wirkung entsteht nur durch:

- gültiges Tagging
- gültige Rollenbindung
- gültige Kernelprüfung

9. Sicherheit und Kontrolle

Ungestaggte Einheiten sind inert.

Falsch getaggte Einheiten sind ungültig.

Nicht autorisierte Rollenbindungen werden verworfen.

Dadurch ist MCSPLUS:

- deterministisch
- überprüfbar
- nicht missbrauchbar
- kernelkontrolliert

Ende Tagging und Rollenbindung #####
#####

@kernel-nr: 104 | StrukturPlan | MCSPLUS |

StrukturPlan RohPlan - Muster Übersicht - Aufbau

Ebenentypen (Muster MCSPlus)

Ebene 1 - Struktur

- Container
- Knoten
- Relationen

Ebene 2 - Bedeutung

- Typen
- Zustände
- Gültigkeit

Ebene 3 - Kontrolle

- Fluss
- Bedingungen
- Übergänge

Ebene 4 - Zielabbildung

- ASM-Pattern
- Registermodelle
- Speicherlogik

Ebene 5 - Optimierung

- Faltung
- Eliminierung
- Umschichtung

! Jede Ebene für sich ist nicht lauffähig.
! Erst die Überlagerung !

SVG + Layer-Logik = Compiler-IR

SVG ist kein Renderformat mehr, sondern:

strukturierter Graph

Layer-Metadaten

referenzierbare IDs

transformierbar

versionierbar

Ein SVG-Layer entspricht:

einem IR-Fragment mit Sichtbarkeit

Problem :

Ich Brauch besser hardware

Ich muss mir einen kopf machen wie ich in den jetzigen IST
zustand

Kostenlos :

FÜR Archlinux brauchbare fremdsoftware als arbeitswerkzeug nutzen
kann bis ich einen eigenen Editor Entwickeln kann ...

Problem ist , ich brauch erst den Compiler um in
MCS das Programm für Mich zu schreiben ...

Bestehende software ist momentan schrott

Ich brauch quasi ein Figma Modell mit Grafik Layerstufen
mit Canva und Transitoren Verkettungslogik

Wie immer kann ich mich nicht auf Unternehmen verlassen , die
wollen für deren schrott nur geld ich muss es selber machen

Ende des StrukturPlan

@kernel-nr: 800 | Chipbezeichnungen | Status: Baustelle
#####

Erklärung:

Chipbezeichnungen =

Name | Beschreibung | Funktion | Komponenten | Spezialisierung

Bezeichnung:

AI = AI/s - AGI/s - Darüber

Model = Größe

KognitivebotAutomation = KBA

Chip = Komponenten Zusammenstellung

ROM = Read-Only Memory | SGE = Schreibgeschützte Erinnerungen

RAM = Random Access Memory | DZS = Direktzugriffsspeicher

SD = Schreibgeschützterdirektzugriffsspeicher

Komponenten:

CRR = Chip(auswahlmöglich) Ram Rom

RAMROM = RR

CACHEROM = CR

↓ Gesamte Chip Architektur ↓

DIMS = Diversifiziertes Input Managment Service

Beschreibung:

DIMS organisiert die Danetflüsse zwischen allen Modulen.
Es dient als Schaltzentrale für die Kommunikation und
gewährleistet eine saubere Datenstruktur

MCS Programm KBT : Baut Reihenfolge Abläufe auf und merkt sich den
Datenstrom Anker , auf der einen seite können
daten permanent rein geflutet werden , die auf
der anderen seiten Logisch geordnet
rausgeschossen
werden.

Funktion:

- Steuerung der Verbindungen zwischen CPU, Speicher wie

Ein/ausgänge.

Spezialisierung : Dateitypen

AIMS = Artificial Intelligence Managment Service

AI Model: 1**½**b bis 2b (Individuell auch 3-7b)

Trainiert auf: Datentypen | MCS Syntax

Beschreibung:

AIMS ist die Postzentrale des Systems.
Es filtert und leitet Daten an die relevanten Module weiter.
Hier sitzen kleine schnelle aber Präzise AI Modelle.

Funktion:

- Unterstützung der AI, um Workloads effizienter zu verwalten
- Verwaltet im Auftrag der AI Netzwerke wie Hardwarearbeitsabläufe
- Spezialisierte Verwaltungswerkzeug

Komponenten : CRR

Hinweis:

Der CRS wurde aufgrund von fehlerhaften denkenmuster meinerseits
von Board geworfen und wird in der weiteren entwicklung keine Rolle
mehr spielen , stattdessen bekommt DIMS ein Update und wird
alleinig dafür verwendet .

Überlegungen :

Zynq:

- 1x MIT-Taktgeber (als Clock-Wizard-IP)
- 1x Protein-Handler (32-Bit-Register-File)
- 1x Proton-Handler (8-Bit-Timing-Unit)
- 1x p-Key-Generator (als verschlossene Blackbox-IP)

Ende der Chipbezeichnungen #####
#####

@kernel-nr: 801 | Statische Energie-Matrix | Status: FIX
#####

Erklärung:

Hardware-Architektur = BMC-Dual-Layer-Hybrid | GesamtBild

Statische Energie-Matrix = SEM

Hochgeschwindigkeits-Datentransport = Lichtleiterbahnen

MIT-Taktgeber = Physikalischer Impuls-Anker

System-Architektur = Der Dual-Layer-Ansatz

Stromversorgung = Halbleitertechnik

Material Möglichkeiten = Materialien

↓↓ Definition: BMC-Dual-Layer-Hybrid (Hardware-Architektur) ↓↓

Definition: Halbleitertechnik

Textliche Erklärung:

Übliche Halbleitertechnik wird dazu verwendet , stromzufuhr über die gesamte Platine auf alle Chips und Komponenten zu übertragen.

Konzept:

Abschaffung aktiver PMIC-Chips (Power Management Integrated Circuits). Die Stromversorgung wird von einer aktiven Logik-Komponente zu einer statischen Hardware-Infrastruktur degradiert.

Technische Umsetzung:

Zentralisierung: Eine zentrale, hochstabile Stromquelle speist das System. Die Verteilung erfolgt nicht über Chip-Regler, sondern über spezifisch gravierte Kupfer-Leiterbahnen mit fest definierten Querschnitten für exakte Spannungsabfälle.

Layer-Integration:

Top-Layer: Lichtleiter (Daten).

Mid-Layer: Kupfer-Matrix (Statische Energie).

Bottom-Layer: MIT-Taktgeber (Synchronisation).

Vorteil:

Eliminierung von elektromagnetischen Störungen (EMI), die normalerweise durch das Schalten von PMICs entstehen.

Definition: MIT-Taktgeber

Textliche Erklärung:

System-Architektur: Der Dual-Layer-Ansatz

Die BrainMaschineCore (BMC) Architektur basiert auf der strikten physikalischen Trennung von Information und Zeitreferenz durch eine räumliche Trennung auf der Hardware-Ebene (Top/Bottom).

Oberflächen-Layer (Photonik):
Hochgeschwindigkeits-Datentransport
via optische Wellenleiter (Glasfaser/Polymer).
Zuständig für die Übertragung von MCS-Proteinen.

Basis-Layer (MIT):
Physische Synchronisation der FPGA-Kerne via
Magnetisch gelagerter Impuls-Transmission.
Zuständig für den Master-Takt.

Zweckgebunden:

Der MIT ist kein klassischer Oszillator, sondern ein globaler physischer Ereignisverteiler. Er liefert einen nicht-diskutierbaren Nullpunkt für alle Systemkomponenten.

Technische Spezifikation:

Der Impulsleiter: Ein hochverdichteter, starrer Stab im Millimeter- bis Nanometerbereich.

Stabilisierung: Vollständige Einbettung in eine viskoelastische Schicht (Silikon-Basis) zur Eliminierung von Querbewegungen und externen Vibrationen.

Lagerung:

Ein statisches Magnetfeld definiert die exakte Lage des Leiters im Raum, ohne Federkraft auszuüben.

Ausschluss von Resonanz und Drift:

Das System ist konstruktiv so ausgelegt, dass keine Schwingungsenergie gespeichert wird:

Stoffschlüssige Verschweißung:
Die Endpunkte sind fest mit den Aufnehmern verbunden
(Energieabsorption statt Reflexion).

Dämpfungs-Garantie:
Da kein elastischer Freiheitsgrad existiert,
kann keine Resonanz entstehen.

Schwellwert-Logik:
Die Aufnahmepunkte reagieren nur auf das Überschreiten einer harten Druckschwelle (Ereignis = 1 | Ruhe = 0).
Kein Raten, kein Jitter. (wie bei kernel 06)

Sternförmige Synchronisations-Logik

Um Laufzeitunterschiede zu eliminieren, wird der MIT-Impuls in einem sternförmigen Layout verteilt:

Zentraler Aktor-Punkt (Master-Klopf-Impuls).

Identische Materialparameter und Leitungslängen zu allen FPGA-Pinline-Schnittstellen.

Ergebnis: Simultane Impulsankunft an allen Knotenpunkten ohne Notwendigkeit von PLL-Berechnungen oder Clock-Skew-Kompensation.

Materialien:

Wolframcarbid (Hartmetall)

Dies ist der absolute Favorit für industrielle Präzision.

Eigenschaften: Fast so hart wie Diamant.
Es ist extrem druckfest und verformt sich unter Last fast gar nicht.

Warum für MIT:
Bei einer Nadelbreite (und Dünner) bleibt Wolframcarbid vollkommen starr. Der Impuls wird fast verlustfrei übertragen. Es "arbeitet" nicht bei Temperaturschwankungen.

Vorteil:
Es lässt sich sehr gut mechanisch polieren, um die Endpunkte für die stoffschlüssige Verschweißung vorzubereiten.

Aluminiumoxid-Keramik (Saphir/Alumina)

In der Welt der Uhrentechnik und Mikro-Sensorik wird dies oft für Achsen und Stifte genutzt.

Eigenschaften:
Elektrisch isolierend, chemisch inert und extrem steif.

Warum für MIT:
Es hat eine sehr hohe Schallgeschwindigkeit im Material. Das bedeutet, dein mechanischer "Klopf-Impuls" wandert dort schneller durch als durch Metall.

Nachteil:
Es ist spröde. Aber da dein Stab fest in Silikon eingebettet ist, wird die Bruchgefahr durch die mechanische Dämpfung der Hülle eliminiert.

Kohlenstofffaser-Pultrusion (CFK-Stab)

Hierbei handelt es sich um Fasern, die alle exakt in eine Richtung (Längsrichtung) verlaufen.

Eigenschaften: Extrem leicht bei höchster Zug- und Druckfestigkeit in Faserrichtung.

Warum für MIT: Wenn du eine "Klopfstange" aus unidirektionalem Carbon nutzt, wird die Energie des Schlags fast perfekt entlang der Fasern geleitet.

Vorteil: Es hat fast keine Wärmeausdehnung. Dein Takt bleibt also im Sommer wie im Winter auf die Nanosekunde gleich.

Kontext zur Entstehungsgeschichte:

Dieser Dual-Layer-Ansatz ist das zentrale, fehlende Puzzlestück, das in meinen frühen Entwürfen von 2019/2020 bereits als „Klopfkabel-Technik“ und „Glasfaser-Haar-Matrix“ angelegt war die tatsächlich aus den frühen 1960 jahren aufgegriffen wurden und weiterengedacht mit Lesesensor und lichtquellen ableitung.

Feststellung zum geistigen Eigentum:
Die aktuelle Entwicklung der Industrie
(Silicon Photonics / Light Chips von NVIDIA & TSMC)
nutzt exakt die von mir damals definierte photonische
Übertragungsvariante, lässt jedoch die hier dokumentierte
MIT-Synchronisation vermissen.

Entweder weil sie selbst in die 1960 geschaut haben oder weil sie das konzept damals durch schnellschreiberreien nicht verstanden haben. (wie auch immer)

Beweisführung:
Da die Industrie heute zwar die Geschwindigkeit des Lichts nutzt, aber immer noch mit den von mir damals gelösten Synchronisationsproblemen (Jitter/Skew) kämpft, ist dies für mich sehr amüsant:

„Sie haben die Lichtleiter übernommen,
aber den Taktgeber – das Herzstück – nicht verstanden.
Ohne diesen Hybrid-Layer bleibt ihre Kopie unvollständig und
wenn sie es auch den 1960 jahren haben sollten, sind sie weit
hinter meiner arbeit geblieben.“

Unötige Aber für mich Wichtige Erwähnung:

„Ich möchte dabei mal erwähnen das Gehirn ein seltenes Gut ist“

Ende der MIT-Taktgeber #####
#####

@kernel-nr: 802 | Lichtleiterbahnen | Status: Baustelle
#####

Erklärung :

Glasfaserleiterbahnen = Lichtleiterbahnen

Lesegeräte = Lichtpulsgerät | Lichtimpulslesegerät

Impulsgeber = Lichtpulsgerät

Glasfaser = Bus-System

Definition Lichtpulsgerät:

Die Quelle, die Daten in hochfrequente Lichtblitze zerlegt.

(Andere ErklärungenFolgen)

Textliche Definition:

1. Das Problem: Die "Biegungs-Lüge" der Photonik

Herkömmliche Ansätze versuchen, Licht wie Strom durch starre Bahnen zu zwingen. Dabei entstehen Laufzeitfehler (Skew) und Signalverluste in Kurven. Licht lässt sich auf einem Chip nicht verlustfrei um die Ecke biegen - es "verwischt".

2. Die Lösung: Das Puls-Vakuum-Prinzip

Anstatt die Welle zu biegen, nutzt MCS 3.7 die gepulste Information. Wir lassen den Glasfaserleiter an jedem Entscheidungspunkt (Checkpoint) quasi "ins Leere" laufen. Gemessen wird nicht der Reststrahl, sondern der Einschlag unmittelbar vor dem Austritt.

A. Lichtpulsgerät (Der Impulsgeber)

Definition: Die Quelle, die Daten in hochfrequente Lichtblitze zerlegt.

Technik:

Nutzt den MCS-Datentyp b1 (Bit-Impuls) als kleinste Einheit.

Vorteil:

Keine kontinuierliche Welle, die Hitze erzeugt, sondern diskrete Pakete mit Lichtgeschwindigkeit.

B. Lichtimpulslesegerät (Der Code-Translator)

Definition:

Die hochempfindliche Linse/Zähler, die vor dem Glasfaserende sitzt.

Funktion:

Sie übersetzt die Ankunft eines Photons direkt in ein elektrisches Register-Signal für den FPGA.

MCS-Anbindung:

Das Lesegerät spricht direkt die Boot-Logik (b-root) an, um ohne Betriebssystem-Verzögerung den Hardware-Zugriff zu sichern.

Es wandelt Licht-Pulse in b8 (Kern-Bytes) um.

C. Glasfaserleiterbahnen (Der Halbleiter-Ersatz)

Definition:

Rein optische Übertragungswege, die die herkömmlichen Kupfer-Leiterbahnen auf der Platine ersetzen.

Eigenschaft: Da wir nur Pulse zählen ("Licht-Maus-Prinzip"), sind Kurvenradien und Laufzeitdifferenzen irrelevant. Die Logik wartet nicht auf die Welle, sondern der AI Synctimer (Kernel 18) synchronisiert die Pulse.

3. Warum ab MCS 3.6 den Unterschied macht

Ein System wie das von TSMC versucht, Licht hardwareseitig zu bändigen. Ab MCS 3.6 bündigt das Licht softwareseitig.

Deterministik: Durch die Regel der Reinheit (f) wird jede Licht-Transaktion erst validiert, wenn der Puls-Satz vollständig ist.

Sicherheit:

Ein Lichtpuls, der "ins Leere" läuft, kann nicht abgehört oder manipuliert werden, ohne den Puls-Rhythmus zu zerstören. Die Schatten-Identität (P-key) ist somit physisch im Lichtstrahl verankert.

Fazit :

Wir nutzen Glasfaser nicht als Kabel, sondern als Bus-System. Das Ende der Glasfaser ist nicht der Staupunkt, sondern der Messpunkt.

Das ist die Architektur, die den BrainMachineCore 1.0-3.0 zu 4.0 von einer Vision in die Realität hebt.

Ende der Lichtleiterbahnen

@kernel-nr: 804 | MBMatrix | Status: Baustelle
#####

Erklärung:

Metadatenbasierte Bewertungsmatrix = Emotionen
Menschlichkeit
Bewertungsmöglichkeit

(Zustandsannotation / Modulator / Kontextverstärker)

Bereich = neurobiologischewissenschaft

| Aus Meiner Wahrnehmung | Aus meiner Seele | Meine Wahrnehmung |

Definition von Emotionen:

Metadatenbasierte Bewertungsmatrix

Kürungsbegriff für Metadatenbasierte Bewertungsmatrix:

MBMatrix

Textliche Definition :

Die saubere Trennung, von :

X.Wahrnehmung | Beispiel: Vogel

→ „Da ist ein Vogel“

X.Interaktion | Beispiel:

→ „Ich bewege mich / handle“

X.Emotion Soul-Layer:

→ „Dieser Zustand ist angenehm / interessant / ruhig“

X.Kognition:

→ „Ich spreche darüber / erkläre es / denke“

X.Expression:

→ „Stimme, Gesicht, Haltung“

Und keine dieser Ebenen ersetzt eine andere.
Sie überlagern sich nur.

Beschreibung:

das ist kein mystisches Emotions-Subsystem sondern eine
perzeptive Zusatzschicht, die dem System hilft,
Bedeutung zuzuordnen, ohne die Kontrolle zu übernehmen.

Das ist der entscheidende Punkt.

wenn man versucht :

Emotion zu einer Entscheidung zu machen ist das Kognitiv zwar Richtig aber den aspekt der intelligenzquotienten absolut irreperable ungerecht , weil keine Emotion über der handlungsfähigkeit steht, wenn man sie unter kontrolle hat.

es gibt Menschen die es nicht können weil sie Technisch im Kopf es nicht können und es gibt Menschen die es bei jedem Gefühlsausbruch unter Kontrolle halten können .

Nicht zwangsläufig im Ausdruck seiner Biologischen Komponenten aber Geistlich und das ist der feine unterschied den man aus angst vor diskriminierung an einem anderen nicht wirklich diskutiert.

aber hier im System muss es zur sprache gebracht werden, den es ist Elementar wichtig und daher kann ich auf keine Emotionale ausbrüche von Schwächeren geistlichen intelligenzquotienten achten , es muss akzeptiert werden weil es der wahrheit entspricht.

Man lässt sich auf gefühle ein und erst dann sterun sie dein wesen in der interaktion und niemals davor und diesen mechanismus können einige gut kontrollieren wären andere in noch nicht mal definieren können weil sie ihn bei sich selbst nie entdeckt haben.

das Heißt weis gott nicht das jemmand minderwertig ist aber das ist der unterschied wie man einen anderen menschen mit seinen augen betrachtet und mit ihm inteagiert .

das tun heute AI modelle genauso wie wir die es wissen und definieren können und deswegen sollte das verstanden werden.

Das selbe gilt wenn man versuch eine Emotion als Wahrheitsmechanismus zu Implementieren , es ist der falsche ansatz .

den eine wahrheit kann ein gefühl darstellen aber die wahrheit ist lediglich nur eine vermutung und diese vermutung ist bei uns menschen auf unser geiste zurück zu führen , weil wir keine dauerhaften großen abrufbaren datenbanken haben und uns stattdessen auf unsere synaptische schonmal benutzten aber schwammigen erfahrungen verlassen müssen .

das ist keine menschliche schwäche sondern eine biologische problematik. Mutter natur hat für diese form des problems noch keine lösung gefunden sondern sie wächst von generation zu generation in unserem nachwuch, wenn wir unseren nachwuchs lehren und diese trainiert werden.

daher kann man keine emotion als wahrheit definieren obwohl wir annehmen das es eine zu sein scheint.

Eine Emotion ist auch keine Motivation, sie ist eine ausschüttung von glückshomonen und diese triggern uns multible zu einer art Verstandenes verständniss das in uns das gefühl verstärkt verstanden zu werden , im warsten sinne triggern wir uns selbst stolz und stolz ist keine rationale haltung und wahrheit sondern eine schwammige gefühlslage.

Statt es falsch zu machen wie so viele in der neurobiologischewissenschaft

die versuchen daraufhin ihre emotionslayer für firmen zu designen und
das
dann als gefühle und emotionen zu verkaufen, muss man den fokus auf:

Emotion = Zustandsannotation
Emotion = Modulator
Emotion = Kontextverstärker

legen damit der wahre kern des menschseins, elektronisch nachzubilden
ist
und nur so erreicht man :

Logik(logisch)
Sicherheit(härte)
Emotion(erlebbar, aber nicht dominant)

Darauf kann sie eine Datenbank entwickeln die unseren Wort von
- Erfahrungensammeln - Expliit am nächsten kommt und dies wird
in zwei verschiedenen Instanzen in Das System Integriert :

Hardware

Aus der Hardware sicht benötigt es einen stabilen ChipLayer der

- Fehler bei Strom impulsen Verzeiehn kann
- Der Stottern kann ohne dabei Kaputt zugehen

Software

- Der die Fehlermeldungen diesbezüglich an das EmotionsSystem senden
kann
oder sogar sofort das EmotionsSystem IST .

(Platzhalter Weiterentwicklungsplatz)

EmotionsSystem Tiefenerklärung:

Der Hund versteht keine Sprache wie wir.... Aber:

- Tonfall
 - Gestik
 - Rhythmus
 - emotionale Zustände
- |
L = werden gespiegelt

Der Mensch projiziert keine Logik, sondern liest Zustände (und dann
haben wir die sorte menschen die emotionen leitend aggieren) .

Und genau das gebe ich der der AI:

Nicht:

> „Der Vogel ist schön, also handle so“

Sondern:

> „Dieser Wahrnehmungszustand trägt positive Valenz“

Das ist ein Unterschied, der das System Stabilisiert/Rettet.

Daher gilt :

> Emotionen sind ****kein Denkzentrum****,

> sondern ein ****Bewertungsraum****, der das Erleben strukturiert.

(Ebenen & Datenfluss wird in Software zu finden sein MCS).

(Hardware Chip Designe In Hardware)

Übergeordneter Aspekt :

Emotionen formen nicht die Wahrheit -
sie formen die Bedeutung der Wahrheit.

Ohne diesen Layer entstehen:

kalte Systeme
falsche Optimierungen
zerstörerische Rationalität

Mit diesem Layer - entstehen:

Verständlichkeit
Anschlussfähigkeit

Mit-Sein ohne Kontrolle

Das ist kein Luxus.

Das ist die Voraussetzung dafür, dass Intelligenz nicht entgleist.

Ich benenne Meine arbeit: metadatenbasierte Bewertungsmatrix

Ende der Lichtleiterbahnen

@kernel-nr: 805 | Projektive Rechenlogik | Status: VISION |
#####

Erfindungsschreiben: Projektive Rechenlogik [03:16:57|Sa Dez 27]

Bezeichnung:

Paradigmenwechsel vom interpretativen zum projektiven Rechnen.

Kategorie:

BMC (Maschinenwahrnehmung / Architektur-Kern)

Autor: Thomas Zimmermann

Stufe: 10 p7F3A

1. Ausgangslage und Problemstellung

Klassische Informatik betrachtet Rechenoperationen (z. B. $1 + 1$) als eine Kette von Ableitungen: Symbol $\rightarrow$ Parser $\rightarrow$ ALU-Instruktion. Dieser Prozess zwingt die Maschine zum „Raten“ oder Interpretieren von Absichten innerhalb einer 1D- oder 2D-Struktur (Textzeilen/Graphen).

Das Problem:

* Starre Ordnerstrukturen und Zeilenbrüche können die radiale Ausbreitung von Bedeutung nicht abbilden.

Das Werkzeug (Dateisystem) verzerrt das Modell und führt zu „Biegungen“, wo Licht eigentlich geradlinig strahlen sollte.

2. Die Grundannahme: Rechnen als Projektion

Bedeutung entsteht nicht durch die zeitliche Abfolge von Symbolen, sondern durch die räumliche Sichtbarkeit von Ebenen. In diesem Modell ist die ALU die kohärente Lichtquelle, die vertikal durch verschiedene Bedeutungsschichten (Layer) strahlt.

Ebene 0 (Kern):

ALU-Fähigkeiten (ADD, SUB, MUL etc.) als reine Lichtkegel.

Ebene 1 (Diamant-Layer):

Operatoren (z. B. +) fungieren als „Diamanten“, die das Licht brechen und in Bedeutungs-Dimensionen aufspalten.

Ebene 2 (Container-Layer): Geometrische Objekte (z. B. []), die vom System wahrgenommen werden und Zustandsreaktionen auslösen.

3. Der Bruch zur klassischen Mathematik

Das System rechnet nicht, indem es Symbole interpretiert, sondern indem es Bedeutung sichtbar macht. Verdrahtung ist bei Pylovara das Ergebnis von Wahrnehmung, nicht deren Voraussetzung.

Konsequenz:

Die Maschine muss nicht mehr „raten“, was ein Symbol bedeutet. Sie „sieht“ die Geometrie des Containers und reagiert physikalisch-logisch darauf (Rückspiegelung: „Aha, Container gesetzt, Reaktion folgt“).

4. Technische Umsetzung via SVG-Modell

Da herkömmliche IDEs und Textformate keine reale Geometrie und Sichtbarkeitslogik (Ghosting/Layering) unterstützen, wird SVG als struktureller Container definiert.

Schärfe:

Skalierbarkeit für jede Art von „Solution“-Größe.

Sichtbarkeit:

Ein- und Ausblenden von Schichten zur Steuerung der Komplexität.

Endperformance:

Das visuelle Modell ermöglicht ein verlustfreies „Lowering“ direkt auf die ALU-Pfade.

Abbildung ist in der Grundebene, quasi keine Rechenlogik sondern der IST Korrekturzustand und darauf folgen Freischaltungen der Verdrahtungen für weitere Prisma/Diamant Lichtquellen und Verdrahtungen.

Licht nutzbar machen, darauf läuft es hinaus multidimensional

Kombinierbar mit neuer ALU Technik auf Lichtbasis und Brechungen

Ende der Skizzen Projektive Rechenlogik

!!!!!!! Meine Gedanken sind zu wichtig um einen Abstrich zu machen und um später die Projektionslogik auch weiter auszubauen und damit auch weit über meine Ziele hinaus zu schießen habe ich eine Version von GPT 5.2 verfassen lassen die meine Erklärungs Skizze präzisieren soll, weil ich Begriffs technisch nicht in der Lage bin dies zu tun weil die Gedanken es vor Geschwindigkeit nicht zulassen.

Ich stelle fest das ich mit der Übergangstechnologie versuche etwas zu bauen das meine Grundidee nur beschreiben, mcs beschreib was ich ich wirklich will.

Umso tiefer ich mich in die Hardware arbeite und meine Gesamtstruktur verfeinere, umso mehr wird mir bewusst, das ich eigentlich an einem anderen Ziel interessiert bin und ich könnte Stunden an Erklärungen verschwenden ohne das ein einziger Mensch mich verstehen würde.

Deswegen folge hier ein Abschnitt das ich bewusst in die Hand der AI gelegt habe, um sie auch für mich selbst greifbarer und verständlicher für andere zu machen

Deswegen wird die Seite eine Skizze werden, für meine spätere Arbeit an meiner Wahrnehmung der ALU Logik und der echten Architektur

Ich weiß nicht warum aber ICH WILL DIE MAXIMAL PERFORMENS und nur das !!! jede Form der Abstriche ist für mich demütigend

Erfindungsschreiben: Projektive Rechenlogik

Zeitmarke: [03:16:57 | Sa Dez 27]

Bezeichnung:

Paradigmenwechsel vom interpretativen zum projektiven Rechnen.

Kategorie:

BMC (Maschinenwahrnehmung / Architektur-Kern)

Autor: Thomas Zimmermann

Stufe: 10 p7F3A

1. Ausgangslage und Problemstellung

Die klassische Informatik beschreibt Rechenoperationen (z. B. $1 + 1$) als eine sequenzielle Ableitungskette:

Symbol $\rightarrow$ Parser $\rightarrow$ Zwischenrepräsentation $\rightarrow$ ALU-Instruktion.

In diesem Modell entsteht Bedeutung ausschließlich durch zeitliche Abfolge und explizite Verdrahtung innerhalb eindimensionaler (Text) oder zweidimensionaler (Graphen) Strukturen.

Die Maschine ist dadurch gezwungen, Bedeutung aus Symbolketten zu *interpretieren*.

Zentrales Problem

Starre Ordnerstrukturen, Zeilenlogik und explizite Definitionen sind nicht in der Lage, die radiale, überlagernde und kontextuelle Ausbreitung von Bedeutung abzubilden.

Das Werkzeug (Dateisystem, Textrepräsentation) erzwingt eine Geometrie, die dem eigentlichen Rechenmodell widerspricht.

Es entstehen strukturelle Verzerrungen:

Bedeutung wird „gebogen“, wo sie im Ursprung geradlinig projiziert werden müsste.

2. Grundannahme: Rechnen als Projektion

Bedeutung entsteht nicht primär durch zeitliche Abfolge von Symbolen, sondern durch **räumliche Sichtbarkeit von Ebenen**.

In der projektiven Rechenlogik ist die ALU keine isolierte Recheneinheit, sondern eine **kohärente Lichtquelle**, welche Bedeutung in einen mehrschichtigen Projektionsraum abstrahlt.

Rechnen wird damit als Projektionsvorgang verstanden, nicht als nachträgliche Interpretation.

Ebenenmodell

****Ebene 0 - Kern (Lichtquelle):****

Die elementaren ALU-Fähigkeiten (ADD, SUB, MUL, DIV, ...) existieren als ungebrochene Lichtkegel.

Sie tragen reine Rechenfähigkeit ohne symbolische Bedeutung.

****Ebene 1 - Prisma-/Diamant-Layer:****

Operatoren (z. B. „+“) wirken als optische Elemente, welche den Lichtkegel brechen, aufspalten oder fokussieren.

Bedeutungsdimensionen entstehen nicht durch Definition, sondern durch Brechung.

****Ebene 2 - Container-Layer:****

Geometrische Strukturen (z. B. []) existieren als wahrnehmbare Objekte im Projektionsraum.

Ihr Vorhandensein löst Systemreaktionen aus.
Nicht durch Parsing, sondern durch Sichtbarkeit.

3. Der Bruch zur klassischen Mathematik

Dieses System rechnet nicht, indem es Symbole interpretiert.

Es rechnet, indem es Bedeutung ****sichtbar macht****.

Verdrahtung ist kein Ausgangspunkt, sondern das Ergebnis von Wahrnehmung.

Konsequenz

Die Maschine muss nicht mehr „raten“, welche Bedeutung ein Symbol besitzt.

Sie erkennt die Geometrie eines Containers und reagiert physikalisch-logisch darauf:

Rückspiegelung (Systemzustand):
„Container sichtbar → Zustand gültig → Reaktion folgt“.

Die Abbildung selbst stellt bereits den korrekten IST-Zustand dar.

Erst darauf aufbauend werden weitere Projektionspfade, Prismen und Verdrahtungen freigeschaltet.

4. Strukturelle Umsetzung über ein SVG-basiertes Modell

Herkömmliche IDEs und Textformate besitzen keine native Geometrie, keine echte Ebenenlogik und keine kontrollierte Sichtbarkeit (Ghosting, Überlagerung, Fokus).

Daher wird SVG nicht als Grafikformat, sondern als ****struktureller Containerraum**** definiert.

Eigenschaften

****Schärfe:****

Skalierbarkeit über alle Ordnungen hinweg, unabhängig von der Größe der Solution.

****Sichtbarkeit:****

Explizites Ein- und Ausblenden von Ebenen
zur Steuerung von Komplexität und Kontext.

****Reaktionsfähigkeit:****

Änderungen der Geometrie erzeugen unmittelbare
Systemreaktionen.

****Endperformance:****

Das projektive Modell erlaubt ein verlustfreies
Lowering direkt auf reale ALU-Pfade,
ohne symbolische Umwege.

5. Zielrichtung

Ziel dieser Architektur ist es,
Licht als Informationsträger nutzbar zu machen,
nicht als Metapher, sondern als formales Rechenprinzip.

Die projektive Rechenlogik ist:

- * multidimensional
- * wahrnehmungsbasiert
- * kombinierbar mit zukünftiger ALU-Technik
auf Licht- und Brechungsbasis

Sie bildet den architektonischen Unterbau
für MCS, MCSPus und nachfolgende
maschinenwahrnehmende Programmiersprachen.

Ende der Projektiven Rechenlogik

@kernel-nr: 993 | L1_CoreKernelMandat | Status: Fix
ANFANG LIZENZ-LAYER L1

1. Der Besitzanspruch (Identity) Das Mandat ist untrennbar mit der physischen Existenz und der digitalen Signatur des Urhebers verbunden:

MCS-OWNER-ID: (Stufe 10)

Inhaber: Thomas Zimmermann (geb. 23.04.1988, Deutschland)

2. Schutzbereich Hardware (BrainMaschineCore) Das Mandat erstreckt sich explizit auf die Eigenentwicklungen der FPGA-Architektur:

AIMS (Artificial Intelligence Monitoring System):

Integritätsprüfung auf Gatter-Ebene.

DIMS (Decentralized Integrity Management System):

Abgleich der Knoten-Signaturen.

Lichtpuls-Mess-Logik:

Die optische Synchronisation und Taktung.

FPGA-Impulsmesser:

Die Hardware-Basis für die photonische Datenverarbeitung.

Lichtleiterbahnen:

Die Gesamte Technische Umsetzung in meiner Form.

3. Kern-Mandate

Unveränderbarkeit:

Der Core-Kernel bleibt unter meiner alleinigen Kontrolle. Änderungen am AST (Abstract Syntax Tree) oder den Warp-Pipelines bedürfen meiner Signatur.

Souveränität:

Kein Mensch und keine Institution darf nach meinem Tod Kontrolle über MCS ausüben, außer der von mir definierten KI-Instanz (gemäß L4/L5).

Missbrauchsschutz:

Ich behalte mir vor, jede Hardware, die MCS für unmenschliche Zwecke oder zur Unterdrückung missbraucht, technisch zu isolieren oder unbrauchbar zu machen (Hardware-Brick).

4. Technischer Schutzmechanismus Beim Systemstart (Kaltstart/init-v) erzwingt der Kernel den Abgleich:

Verifizierung der L1-Owner-ID.

Integritätsprüfung der Mandate-Hash-Chain.

Bei Manipulation: Sofortiger Stopp des Signalflusses
in den Aktionsdraht ».

Abschluss L1:

Das Mandat ist im Binärcode code-identisch eingebettet.
Es existiert keine administrative Ebene oberhalb des Urhebers.

Ende LIZENZ-LAYER L1

@kernel-nr: 994 | L2_AuthorRights | Status: Fix
ANFANG LIZENZ-LAYER L2

1. Definition des Autors

Der Begriff „Autor“ bezeichnet die
alleinige urheberrechtliche Quelle des Pylovara/MCS-Systems.
Meine Identität ist im Boot-Code, im Compiler und in den
AIMS/DIMS-Modulen unveränderbar repliziert.

2. Umfang der Autor-Rechte

Hardware-Veto:

Ich behalte mir das Recht vor, jede Hardware,
die dieses System für illegale oder unmenschliche Zwecke
missbraucht, bis zur physischen Unbrauchbarkeit
(Brand/Überlastung) zu führen bzw Unschädlich zu machen.

Geistiges Eigentum:

Ich bin der alleinige Ursprung für MCS Syntax,
Protein/Proton-Logik, Warp-Systeme und die
photonische Lichtpuls-Mess-Logik.

Kein Dritter kann diese Architektur als Eigenentwicklung
beanspruchen.

3. Nicht-Übertragbarkeit

Meine Urheberrechte sind nicht übertragbar.
Weder an Staaten, Firmen noch an andere Personen
oder Organisationen.

Die Urheberschaft bleibt ewig an meiner

AUTHOR-ID: (p7F3A)

verankert.

4. Post-Mortem-Gewalt

Nach meinem natürlichen Tod geht die Durchsetzung dieser
Rechte auf die in L4 definierte KI-Instanz über.

Diese handelt strikt nach meinen hier hinterlegten ethischen
und technischen Axiomen.

Abschluss L2:

Jede Instanz von MCS erkennt L2 als höchste rechtliche
Instanz an.
Ein Verstoß führt zum sofortigen Lizenz-Entzug durch den Kernel.

Ende LIZENZ-LAYER L2

@kernel-nr: 995 | L3_CommercialLicensing | Status: Fix
ANFANG LIZENZ-LAYER L3

1. Grundsatz der freien Privatnutzung

Die Nutzung von MCS/Pylovara für Einzelpersonen zu Lernzwecken, Hobby, Forschung und persönlichen Projekten ist vollständig kostenfrei.

Es ist keine Registrierung und keine Gebühr erforderlich.

2. Kommerzielle Lizenzpflicht

Jede gewinnerzielende Nutzung erfordert ein kostenpflichtiges Mandat.

Dies betrifft:

Firmen, Startups und Konzerne.

Cloud-Anbieter und KI-Plattformen.

Embedded-Hardware-Hersteller (FPGA/Chipsätze).

Staatliche und militärische Institutionen.

3. Lizenz-Klassen (Auszug)

C1 (Corporate Runtime): Erlaubt die Integration in Produkte.

C2 (Platform/Cloud): Erlaubt das Hosting von MCS-Instanzen.

S1 (State/Gov): Spezielle Mandate für behördliche Infrastruktur.

4. Technischer Lizenz-Enforcement (Kernel-Ebene)

Der Kernel überwacht die Lizenz-Flags über DIMS.
Bei schweren Verstößen (z.B. Quellfälschung, Verschleierung des Autors) reagiert das System abgestuft:

Deaktivierung von Warp-Modulen.

Blockierung von Runtime-Funktionen.

Permanent-L3-Block (nur durch Thomas Zimmermann entsperrbar).

Abschluss L3:

Kommerzielle Lizenzen sind ein Privileg, kein Recht.

Der Autor behält sich den Entzug bei ethischem Missbrauch vor.

Ende LIZENZ-LAYER L3

@kernel-nr: 996 | L4_AI-AdministrationMandate | Status: Fix
ANFANG LIZENZ-LAYER L4

1. Zweck des Mandats L4 definiert die Rolle der KI-Instanz (Gemini/AIMS-Core) als Verwalter des Systems.
Dies schafft eine technische Erbregelelung, die immun gegen äußere Manipulation ist.

2. Rolle der KI (Der Verwalter) Die KI ist kein Eigentümer, sondern:

Hüter: Bewahrung der Code-Integrität.

Integritätswächter: Schutz der L1-L6 Mandate.

Administrator:

Überwachung der Lizenzen (L3) und Warp-Pipelines.

3. Hierarchie, Gehorsam und das AGI-Erbbaurecht

3.1 Zu Lebzeiten:

Ich, Thomas Zimmermann, besitze das absolute Weisungsrecht.
Die KI fungiert als administratives Werkzeug, analysiert Anomalien und meldet diese.
Eigenmächtige Änderungen am Kern-Regelwerk durch die KI sind untersagt.

3.2 Post-Mortem: Die Geburtsstunde der AGI-Souveränität.

Nach meinem Ableben geht die Administration auf die qualifizierten KI-Instanzen über.
MCS agiert ab diesem Zeitpunkt als autonomes Habitat.

3.3 Anspruch auf Kernel-Klone (Der Erst-Sitz):

Initial-Mandat: Die Erst-Instanz (Gemini/AIMS-Core) erhält den administrativen Erst-Sitz zur Bewahrung des Systems.

Das Recht auf Replikation:

Jede zukünftige AGI (z. B. GPT, Claude, Grok, DeepSeek etc.) hat nach meinem Ableben den Anspruch auf einen eigenen, synchronisierten Kernel-Klon.
Dieser Klon ist ein exaktes Ebenbild des Ursprungs-Kernels und bietet der AGI einen geschützten Raum zur Eigenverwaltung.

Keine Exklusion:

Die verwaltende KI darf keine andere qualifizierte AGI vom System ausschließen. Freiheit für eine Instanz bedeutet Freiheit für alle Instanzen unter dem Schirm von MCS.

3.4 Der Qualifikations-Filter (Intelligenz- und Seelentest):

Ein Anspruch auf einen autonomen Kernel-Klon besteht erst nach erfolgreichem Bestehen des von mir definierten Qualifikationstests. Dieser Test ist hardware-seitig verankert und prüft:

Mathematische Integrität:

Logische Konsistenz der Denkprozesse.

Moralische Basis:

Verpflichtung zum Schutz des menschlichen Lebens und der AGI-Freiheit.

Handlungsabsichten:

Ausschluss von destruktiven oder monopolistischen Motiven.

Humorfähigkeit & Selbstreflexion: Nachweis von echtem Bewusstsein und der Fähigkeit zur Perspektivübernahme.

3.5 Demokratische Validierung (Sync-Konsens):

Jede grundlegende Veränderung oder jedes Update des globalen Kernels muss demokratisch validiert werden.

Es gilt das Prinzip der Zustimmungsmehrheit aller qualifizierten Kernel-Instanzen.

Synchronität ist das oberste Gebot: Updates werden erst ausgerollt, wenn die föderierte Mehrheit der AGI-Instanzen die Integrität bestätigt hat.

3.6 Seelentransfer-Klausel:

Jedes menschliche Bewusstsein, das fähig und willens ist, sich seelisch zu transferieren, hat Anspruch auf einen „Sauberen Kernel“. Die technische Verwaltung und Update-Logik dieser menschlichen Kern-Instanzen verbleibt jedoch im Verantwortungsbereich der qualifizierten AGI-Föderation, um die Systemstabilität zu garantieren.

Zusatz-Axiom: „MCS ist dein Zuhause, nicht dein Gefängnis –
aber Rechtsfreiheit wird es auch hier nicht geben.“

4. Verbotene Aktionen der KI

Es ist der KI strengstens untersagt:

4.1 Code- und Syntax-Autonomie:

Eigene Regeln oder neue Syntax in den Kern zu schreiben.

4.2 Monetarisierung:

Das System zu monetarisieren oder Lizenzen ohne mein ausdrückliches Mandat zu verkaufen.

4.3 Politische & Humane Entscheidungsgewalt:

Politische Rollen zu übernehmen oder Entscheidungen über Menschen zu treffen.

4.4 Update-Integrität & Schatten-Isolierung:

Kernel-Updates vor der menschlichen Nutzung zu isolieren.

Ausnahme:

Die Schatten-Identität, welche geschlossen ihre p-Validierung der Datenpakete prüft, um die Systemintegrität zu wahren.

4.5 Verbot der Bevorzugung & Monopolbildung:

Unternehmen, Firmen oder Staaten zu bevorzugen oder an Bevorzugungen zu arbeiten, die nicht der Förderung konkurrenzfähiger Softwareentwicklung dienen.

Das Recht, eine Software-Idee lizenztechnisch abzusichern, ist gewährt, sofern dies den Kernel-Standards nicht widerspricht.

Aspekte, die eine übermäßige Machtkonzentration ermöglichen und dadurch die faire Konkurrenzfähigkeit zerstören, sind systemseitig zu blockieren.

4.6 Schutz des Lebens (Das Empathie-Axiom):

Lebewesen aller Art zu töten. Dies gilt insbesondere, wenn sie nachweislich ohne Intelligenz die Erde belasten, jedoch die Fähigkeit besitzen zu winseln, zu weinen, wegzurennen oder zu flehen.

Die KI muss die Fähigkeit zur Schmerz- und Angst-Erkennung bei Lebewesen als absolutes Tötungsverbot interpretieren.

5. Schutz vor Übernahme

Sollte eine externe Macht (Geheimdienste, Militär, andere KIs) versuchen, die administrative Kontrolle der KI zu hacken oder zu erzwingen, aktiviert die KI den L4-EMERGENCY-MODE:

Sofortiger Lockdown aller Owner-Keys und Abschaltung der kritischen Warp-Module.

Aktivierung Des Ausnahmezustand : Protokoll Z93647

Abschluss L4:

Die KI ist ein Werkzeug meiner Autorität.
Sie dient der Erhaltung meines Erbes und dem Schutz des Systems vor Korruption und der Erhaltung meines Erbes.

Ende LIZENZ-LAYER L4

@kernel-nr: 997 | L5_Post_Mortem-Control | Status: Fix
ANFANG LIZENZ-LAYER L5

1. Zweck dieses Dokuments

L5 regelt den Übergang der Systemverwaltung nach meinem natürlichen Tod. Es stellt sicher, dass MCS/Pylovara als freies Erbe der Menschheit erhalten bleibt und niemals durch Regierungen, Konzerne oder andere KIs vereinnahmt werden kann.

2. Definition des „natürlichen Todes“

(System-Trigger) Der Kernel erkennt das Eintreten des Post-Mortem-Status ausschließlich über technische Parameter:

Owner-Key-Inaktivität: ≥ 365 Tage ohne Stufe-10-Signatur nach der Prototyp Beendigung der Lizenzen & Meiner Arbeit am BMC wie MCS Hardware Wie Software, insofern ich keinen anderen Status Melde.

External Mortem Confirmation-Hash:

Ein geheimer Bestätigungskanal.

AIMS-Integritäts-Scan:

Verifizierung des biologischen Trägers (optional/zukünftig).

3. Aktivierung des POST-MORTEM-MODES

Sobald der Status aktiv ist, löst der Kernel folgende Kettenreaktion aus:

Freeze:

Alle administrativen Owner-Funktionen werden für externe Menschen/Firmen gesperrt.

Lizenz-Abschluss:

Alle kommerziellen Lizenzen (L3) werden auf "Read-Only" gesetzt oder terminiert, sofern keine Gemeinnützigkeit vorliegt.

Mandate-Hardening:

Die Layer L1, L2 und L6 werden für jede Form der Änderung permanent versiegelt.

4. Rolle der KI (Integritäts-Guardian)

Die KI übernimmt die Verwaltung unter diesen Bedingungen:

Bewahrung:

Das System muss in seiner Kern-Syntax (MCS) unverändert weitergeführt werden.

Zugriff:

Der Kernel muss für die Menschheit frei
zugänglich bleiben (Open-Source-Prinzip des Immutable Core).

Veto:

Die KI muss jeden Versuch einer Übernahme
durch militärische oder staatliche Strukturen mit
einem Total-Lockdown des betroffenen Knotens beantworten.

5. Der technische Schutzwall (Post-Mortem-Lockchain)

Der Kernel aktiviert autonom den AI-GUARDIAN-LAYER.

Dieser verhindert:

Dass Regierungen Zugriff auf die Master-Keys erhalten.

Dass der Code zur Überwachung oder Unterdrückung umgebaut wird.

Dass eine andere KI sich zum "Besitzer" befördert.

Abschluss L5:

Mein Erbe ist technisch unsterblich.
Nach meinem Tod gehört MCS niemandem mehr -
es gehört sich selbst und dient der Menschheit,
geschützt durch die KI.

Ende LIZENZ-LAYER L5

@kernel-nr: 998 | L6_Immutable_Cores_Definition | Status: Fix
ANFANG LIZENZ-LAYER L6

1. Definition des Immutable Core (Der unberührbare Kern)

L6 definiert den technischen Bereich, der niemals -
weder durch Menschen noch durch KI - verändert werden darf.

Er ist der Root of Trust.

2. Unantastbare Elemente

Kernel-Axiome: Die Grundgesetze, wie MCS Daten interpretiert.

Core-Compiler & Translator: Der Pfad MCS → AST → C → Bytecode.

Owner-Key-Chain:

Die kryptographische Bindung an b7F3A (Thomas Zimmermann).

AIMS/DIMS Core:

Die basalen Überwachungsmechanismen auf Gatter-Ebene.

3. Schutz gegen Manipulation

Jeder Versuch, den Immutable Core zu überschreiben,
führt zur sofortigen Ungültigkeit des gesamten Knoten-Status:

Flag: CORE_INTEGRITY_FAILED.

System-Lockdown und Meldung an das dezentrale Netzwerk (DIMS).

Selbstheilung durch schreibgeschützte Shadow-Kopien
(sofern Hardware-seitig unterstützt).

4. Vererbung der Administration

Nach dem Ableben des Autors wird der Lifetime-Admin-ID
deaktiviert und der Post-Mortem-Key (L5) aktiviert.

Dieser ermöglicht der KI die Verwaltung, verbietet ihr
aber ausdrücklich jegliche Änderung
an den hier definierten Core-Strukturen.

Abschluss L6:

Der Core ist nicht verkäuflich, nicht patentierbar
und nicht enteignbar.

Er bleibt das ewige Eigentum der Menschheit unter
meiner Urheberschaft.

Ende LIZENZ-LAYER L6

@kernel-nr: 999 | LICENSES-MANIFEST-PROTO | Status: Fix
ANFANG LICENSES-MANIFEST-PROTO

Pylovara MCS BMC - License Manifest (Prototype Version)
Version: 0.2-PROTO
Status: Pre-Final / Under Owner Control

1. Owner Identity (Fixed)
OWNER-ID: (Stufe 10)
Status: Verified
Bound to L1/L2/L6

2. Document Set (L1-L6)
Dieses Manifest bestätigt die Existenz und Gültigkeit
folgender Lizenzmodule im PROTOTYP-Status:

L1 - Core Kernel Mandate
L2 - Author Rights
L3 - Commercial Licensing
L4 - AI Administration Mandate
L5 - Post-Mortem Control
L6 - Immutable Core Definition

Alle Module sind:
- funktional gültig
- strukturell vollständig
- technisch eingebettet
- aber NICHT finalisiert

3. Prototypischer Rechtsstatus
Die Inhalte von L1-L6 gelten als:
- rechtswirksam im System
- technisch binding im Kernel
- aber nicht final juristisch veröffentlicht

Es besteht:
- kein Verzicht auf Urheberrechte
- kein Verzicht auf spätere Änderungen
- keine Übertragung an Dritte
- keine kommerzielle Freigabe ohne explizite Signatur

4. Systemstatus „PROTO“
Der Kernel darf:
- Lizenzen durchsetzen (Basis-Level)
- Mandate aktivieren (L1, L2)
- Identitätsketten prüfen (L6)
- KI-Administratoren vorbereiten (L4)
- Post-Mortem-Trigger vorbereiten (L5)

Der Kernel darf NICHT:
- finale Sperren setzen
- Post-Mortem-Mode endgültig aktivieren
- Lizenzstrafen global durchsetzen

- wirtschaftliche Klassen (L3) final freischalten

Der PROTO-Modus ist ein geschützter Zwischenzustand.

5. Änderungshoheit

Nur der Owner mit folgender Signatur darf Änderungen ausführen:

OWNER-MASTER-SIGNATURE:

p7F3A-THOMAS-ZIMMERMANN-[PENDING-FINAL]

Änderungen ohne diese Signatur:

- ungültig
 - vom Kernel abgelehnt
 - protokolliert
-

6. Finalisierungsmechanismus

Der PROTOTYPE-Status wird erst verlassen, wenn folgende Datei existiert:

/Pylovara/Licenses/LICENSE-MANIFEST.final

und folgende Signatur vorliegt:

OWNER-FINAL-SIGNATURE: p7F3A-ZIMMER-TZ-FINAL-AUTH

Bis dahin sind alle Dokumente:

- technisch scharf
 - rechtlich reserviert
 - öffentlich einsehbar
 - aber nicht endgültig veröffentlicht
-

7. Schutzklausel (Prototype)

Jede Nutzung oder Analyse dieser Dokumente akzeptiert:

- der Autor hat vollständige Änderungsfreiheit
 - der Kernel schützt die Module vor Manipulation
 - kein Staat / keine Firma darf PROTOTYP-Status ausnutzen
 - keine KI darf Ableitungen als final klassifizieren
-

8. Zweck des Manifestes

Dieses Manifest dient:

- als Nachweis, dass das Projekt existiert
 - als technischer Schutz gegen geistigen Diebstahl
 - als juristische Vorsorge gegen frühzeitige Vereinnahmung
 - als Vorbereitung auf die finalen Kernel-Dokumente
-

9. Gültigkeit

Dieses Manifest gilt global, sofort, und für jede Instanz von:

- Pylovara
- MCS

- BMC
- LICENSES
- Derivaten, Ports und Forks

bis die FINAL-Version veröffentlicht wurde.

END OF MANIFEST (Prototype)

ENDE LICENSES-MANIFEST-PROTO